

Hochschule Hannover

Bachelorarbeit : UI für SOAAP-System-2.0

von

Autor: Yifan Wang

Matr.Nr.: 1571821

Studiengang: Elektrotechnik und Informationstechnik

Vertiefung/Schwerpunkt: Automatisierungstechnik/Prozessinformatik

Erstprüfer: Prof.Dr.-Ing. Martin Streitenberger

Zweitprüfer: Prof.Dr.-Ing. Robert Patzke

von der

Fakultät I

Elektro- und Informationstechnik

24. August 2026

Vorwort

Die Arbeiten werden vom 8. Dezember bis zum 8. März ausgeführt und erstrecken sich über einen Zeitraum von drei Monaten. Ich möchte meinen Dank an all jene richten, die mir Unterstützung und Hilfe zukommen ließen.

Zunächst einmal möchte ich meinem Erstbetreuer Herrn Streitenberger für seine wertvolle Unterstützung während der gesamten Zeit und seine konstruktiven kommentare und Anregungen für meine Arbeit danken. Ich bedanke mich auch bei meinem Zweibetreuer herr Patzke für seine geduldige Anleitung vom seinen Program SOAAP und Hilfe.

Mein Dank geht acuh an herr Mennecke für die Hilfe, die mir gewährt hat, und an den Kolleginnen oder Kollegen, die ich im Labor getroffen habe, für ihre gegenseitige Unterstützung.

Inhaltsverzeichnis

List of Figures	iv
List of Tables	vi
Abkürzungsverzeichnis	vii
1 Einleitung	1
2 Theoretische Grundlagen	3
2.1 Andorid	3
2.1.1 Android Studio	3
2.1.1.1 API/SDK	4
2.2 Eingebettete Systeme und Mikroprozessor/Mikrocontroller	5
2.2.1 Arduino	5
2.2.2 Esp8266	7
2.3 Kommunikationstechnologien	7
2.3.1 OSI-Modell und TCP/IP-Protokollstruktur	8
2.3.1.1 Socket	10
2.3.2 WiFi-Kommunikation	12
2.3.2.1 Access Point	12
2.4 Serielle Schnittstelle - UART	13
2.5 MIDI	14
2.5.1 MIDI Verbindungen	14
2.5.2 MIDI-Interface	16
2.6 Eulerwinkel	17
3 Technisches Umfeld und theoretischer Teil der Arbeit	19
3.1 Android Programmierung	19
3.1.1 JAVA	19
3.1.2 Thread Programmierung	19
3.1.3 UI-Komponente	21
3.1.4 Android Klasse für Stream Socket	21
3.2 Mikrocontroller	23
3.2.1 Arduino Nano 33 BLE	23
3.2.2 Wemos D1 mini pro	24
3.2.3 Esp8266 Arduino Core	25

4	Über SOAAP	29
4.1	Was ist SOAAP	29
4.2	Projekthintergrund SOAAP	30
4.3	Konzept des SOAAPs	31
4.3.1	Demonstrationsaufbau des SOAAP-Systems	31
4.3.2	MIDI-USB-Umsetzer	33
4.3.3	Klassen im Master	35
4.4	Eulerwinkel und MIDI-Value im SOAAP	37
4.5	Klasse MeasMuse	40
4.5.1	Übersicht	40
4.5.2	Mapping Algorithmen	40
4.5.3	Parameter in der Klasse <i>MeasMuse</i>	42
4.5.4	eingebettete Klasse <i>Posture2Midi</i>	44
4.5.5	Konfigurationsfunktion und Anwenderfunktion	47
5	Entwicklung des Userinterfaces	49
5.1	Systemkonzept des Userinterfaces	49
5.1.1	Motivation des Userinterfaces	49
5.1.2	Problemstellungen und Anforderungen des Userinterfaces	49
5.1.3	System Schematik	51
5.1.4	Prozesse Erklärung	52
5.1.5	Protokolle für Userinterface	53
5.1.6	Verlauf der Daten	58
5.2	Android Applikation	59
5.2.1	Übersicht	59
5.2.2	Codeerklärung	60
5.2.2.1	SOAAPClient	60
5.2.2.2	BottomSheetDialog	70
5.2.2.3	MainActivity	70
5.2.3	Darstellung des Layouts	72
5.3	Esp8266 Programmierung	74
5.3.1	Übersicht	74
5.3.2	Codeerklärung	74
6	Zusammenfassung und Ausblick	77
7	Anhang	79
	Literaturverzeichnis	82

Abbildungsverzeichnis

1.1	SOAAP-Projekt	2
2.1	OSI-Modell und TCP/IP-Modell	8
2.2	Datenaustausch mit der Socket-Funktion unter TCP und UDP [1]	11
2.3	Access Point	13
2.4	Datenaustausch der UART Kommunikation [2]	13
2.5	Telegramm der UART Kommunikation [2]	14
2.6	Aufbau des 5-pin DIN-Kabel	15
2.7	Schaltplan der DIN-Buchsen IN/THRU und OUT	16
2.8	MIDI-Interface Produkte	17
2.9	MIDI-Interface MIDI2x2 von MidiPLUS	17
2.10	Roll-, Pitch- und Yawwinkel vom Eulerwinkel	18
3.1	Vorderansicht des Arduino Nano 33 BLE [3]	23
3.2	Vorderansicht des Wemos D1 mini pro [4]	24
4.1	Demonstrationsaufbau des SOAAP-Systems [5]	31
4.2	Datenschema des SOAAP-Systems	32
4.3	Umbaustruktur des Arduinos und MIDI-Interfaces	33
4.4	Vorderseite des MIDI-USB-Umsetzer	34
4.5	Rückseite des MIDI-USB-Umsetzer	35
4.6	Im Master verwendete Klassen	35
4.7	Platzierung von dem Slave und der Hand	37
4.8	Zusammenhang zwischen der Richtung der Handaktivität und dem Eulerwinkel	38
4.9	List der 14 Notenlängen	39
4.10	Graph der lineare Funktionsgleichung	41
5.1	Schematik des Userinterfaces und des SOAAP-Systems	51
5.2	Laufende Prozesse in Arduino Nano 33 BLE, Esp8266 und Android	52
5.3	Laufende Prozesse in Esp8266 und Android	53
5.4	Vereinfachter Schematik des Userinterfaces und des SOAAP-Systems	54
5.5	Aufbau des Telegramms mit TCP-Protokoll	54
5.6	Aufbau des PDUs	55
5.7	drei Handshakes der TCP-Protokolls	55
5.8	TCP-Pakete mit nützliche Daten	56
5.9	TCP-Pakete Details	57
5.10	Folge TCP Stream	57
5.11	Verlauf der Daten zwischen dem Master und der Android-Applikation	58

5.12 Zustandsdiagramm der Erstellung der TCP-Kommunikation	65
5.13 Zustandsdiagramm der Erstellung der TCP-Kommunikation (implemen- tierter Teil)	66
5.14 Prozess der Methode <i>creatPDU()</i>	67
5.15 Threads in <i>MainActivity</i>	71
5.16 Kontrollleiste im Layout der Android-Applikation	72
5.17 Bottom Sheet im Layout der Android-Applikation	72
5.18 Rangesliders und Sliders Layout	73
5.19 Toggle Buttongroup Layout	74
5.20 Prozessablauf im <i>loop()</i> (<i>Esp8266</i>)	75

Tabellenverzeichnis

2.1	Pin-List des 5-pin DIN-Anschluss	15
3.1	Methoden der Klasse <i>InputStream</i>	22
3.2	Methoden der Klasse <i>OutputStream</i>	22
3.3	Arduino Nano 33 BLE Tech specs [3]	24
3.4	Wemos D1 mini pro Tech specs [4]	25
5.1	Formulierung des Integer-Array <i>PduintArray</i>	69
5.2	Umwandelung vom Datentype <i>Meas2Midi</i> zu Integer	69
5.3	Umwandelung vom Datentype <i>MeasMap</i> zu Integer	69
5.4	Umwandelung vom Datentype <i>MidiOpMode</i> zu Integer	70
5.5	Formulierung des Float-Array <i>PdufloatArray</i>	70
5.6	Formulierung des Byte-Array <i>PduByteArray</i>	70
5.7	UI-Komponenten und die gesteuerte Parameter	71

Abkürzungsverzeichnis

SOAAP	Steuerung optischer und akustischer Ausgaben für Performance-Künstler
MIDI	Musical Instrument Digital Interface
UI	Userinterface
GUI	Graphical User Interface
API	Application Programming Interface
SDK	Software Development Kit
WLAN	Wireless Local Area Network
OSI-Modell	Open Systems Interconnection - Modell
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
IP	Internet Protocol
AP	Access Point
UART	Universal Asynchronous Receiver-Transmitter
Cpp	C plus plus Programming Language
BLE	Bluetooth Low Energy
IDE	Integrated Development Environment
IMU	Inertial Measurement Unit

Kapitel 1

Einleitung

Die Technologie hat die Entwicklung der Musik vorangetrieben. Vor der technologischen Revolution war die Musik sehr homogen, die Instrumente waren in Bezug auf Typ und innere Struktur noch nicht perfektioniert, und der Produktionsprozess war unkompliziert, was zu weniger perfekter Musik führte. Es gab keine Werkzeuge, um Musik aufzunehmen, und die Menschen konnten Musik nur in der Kirche oder im Theater hören. Nach der ersten industriellen Revolution erlebten die Musikinstrumente einen großartigen Wandel. Und in dieser Zeit wurde eine große Anzahl von Instrumenten wurde geschaffen und umgestaltet. In den 1870er Jahren erfand der amerikanische Erfinder Thomas Edison den ersten Phonographen, der es ermöglichte, Musik auf eine völlig neue Art und Weise zu übertragen, nicht nur mit Musikinstrumenten oder der menschlichen Stimme. In der heutigen Zeit, mit der fortschreitenden Entwicklung der Computertechnologie, sind Musikproduktion und -vertrieb vollständig digital geworden. Einige Avantgarde-Musiker oder Elektronik-Freaks experimentieren ständig mit neuen musikalischen Installationen und Performance-Formen. Das *SOAAP*-Projekt wurde von vier herausragenden Professoren der Fachhochschule Hannover ins Leben gerufen, mit dem Ziel ein intelligentes Gerät zu entwickeln. Dieses innovative Gerät ermöglicht es, Musik und Licht auf der Grundlage moderne Mikrocontroller- und Sensortechnologie durch menschliche Körperbewegungen zu steuern.

Das Ziel dieser Arbeit war es, einen geeigneten UI (Userinterface) zu entwickeln, da der UI eine wichtige Brücke zwischen Mensch und Maschine darstellt und ein wesentlicher Bestandteil der Mensch-Maschine-Interaktion ist. Dabei wurde das UI über WiFi mit

dem SOAAP-System kommuniziert, indem das Esp8266-WiFi Modul verwendet wurde. Das GUI wurde mit einer Android-APP entwickelt und es wurden folgende Ziele erreicht: Die Kommunikation zwischen Android und Esp8266-WiFi Modul erfolgte durch bestimmte Protokolle und eine fertige GUI wurde erstellt.

Bei dieser Arbeit wurden die folgenden Ziele erreicht:

- Kommunikation erfolgt von Android zum Esp8266-WiFi Modul
- Kommunikation erfolgt durch bestimmte Protokolle
- fertige GUI erstellen

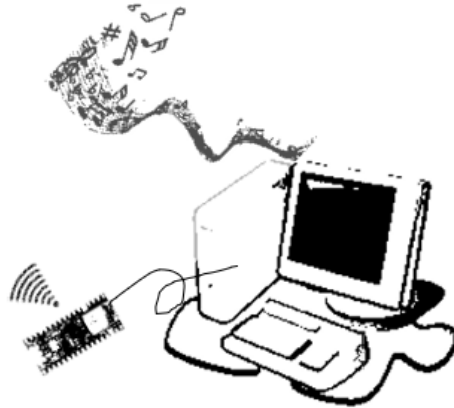


ABBILDUNG 1.1: SOAAP-Projekt

Kapitel 2

Theoretische Grundlagen

2.1 Andorid

In diesem Projekt wird der Userinterface über Android entwickelt, deswegen wird in diesem Abschnitt erklärt, was Android ist. Bei Android handelt es sich um ein mobiles Betriebssystem, das vom Suchmaschinengiganten Google stammt und ist seit dem 21. Oktober 2008 offiziell verfügbar. Als Betriebssystem wird die Software verstanden, die dem Nutzer die Bedienung des Smartphones oder Tablets erlaubt. Sowie etwa Windows auf Computern genutzt wird, ist Android das gängigste Betriebssystem für Smartphones und Tablet. Seit neuerem ist es auch für Fernseher und sogenannte Wearables verfügbar. Es steht dabei in Konkurrenz zu den mobilen Betriebssystemen IOS von *Apple* und Windows Phone von *Microsoft*. Allerdings verwendet die überwiegende Mehrheit der auf dem Markt befindlichen Mobiltelefonhersteller (z.B. SAMSUNG, MI, Google usw.) Android.

2.1.1 Android Studio

Android Studio ist die offizielle integrierte Entwicklungsumgebung (IDE) für die Entwicklung von Android-Anwendungen. Sie basiert auf IntelliJ IDEA, einer integrierten Java-Entwicklungsumgebung für Software, und enthält deren Codebearbeitungs- und Entwicklerwerkzeuge. Um die Anwendungsentwicklung innerhalb des Android-Betriebssystems zu unterstützen, verwendet Android Studio ein Gradle-basiertes Build-System, einen Android-Emulator, Code-Vorlagen und die Integration von GitHub. Jedes Projekt in

Android Studio hat eine oder mehrere Modalitäten mit Quellcode und Ressourcendateien. Diese Modalitäten umfassen Android-App-Module, Bibliotheksmodule und Google App Engine-Module. Zur Erstellung der Software, die Gegenstand dieser Arbeit ist, wurde ebenfalls das Programm *Android Studio* verwendet.

2.1.1.1 API/SDK

Ein *Software Development Kit (SDK)* [6] besteht aus mehreren Tools, die typischerweise vom Hersteller einer Hardwareplattform, eines Betriebssystems oder einer Programmiersprache bereitgestellt werden. Das SDK für Android heißt *Android Software Development Kit* (kurz *Android SDK*), was eine Entwicklungsumgebung und Sammlung von Informationen und Programmbibliotheken, die zur Erstellung von Software für ein bestimmtes Betriebssystem notwendig sind.

Eine *Anwendungsprogrammierschnittstelle (Application Programming Interface-API)* [7] ist ein bestimmter Satz von Regeln zum Code und Spezifikationen, denen Programme folgen können, um miteinander zu kommunizieren. Es verbindet Soft- und Hardwarekomponenten, beziehungsweise sie übersetzen die Software in Maschinsprache und machen sie so für andere Komponenten lesbar. Einzelne Programmteile/Module werden vom eigentlichen Code abgekapselt. Die Module kommunizieren dann ausschließlich über die Schnittstelle (API).

In aktuellen Projekten der Android-Entwicklung stellt sich oft die Frage, ob die entwickelte App auf dem Zielgerät funktioniert, ob die App mit dem Zielgerät kompatibel ist, deswegen muss die richtige *SDK-Version* und *API-Level* (*SDK-Version* und *API-Level* stellen eine iterative Aktualisierung des Programms dar) wählen. *SDK-Versions-Update* und Steigung des *API-Levels* sind Fehlerkorrekturen und neue Funktionen, die zu älteren Versionen des Programms hinzugefügt werden. So sorgt eine ältere *SDK-Version* und *API-Level* zwar dafür, dass die Software auch auf älteren Versionen von Android und damit auf deutlich mehr Endgeräten läuft, sie bieten jedoch auch weniger Funktionen an. Für Entwickler ist es daher wichtig, den Versionsstand gegen den Umfang der Anwendung abzuwägen. Die niedrigere Version wird für dieses Projekt gewählt, da die Funktionalität der höheren Version in diesem Projekt nicht benötigt wird und der Anwendungsbereich ganz oben auf der Liste steht.

2.2 Eingebettete Systeme und Mikroprozessor/Mikrocontroller

Ein eingebettetes System ist eine Kombination aus Hardware und Software, ein Gerät, das unabhängig arbeiten kann und dazu bestimmt ist, eine oder mehrere spezifische Funktionen innerhalb eines größeren Systems auszuführen. Diese Systeme können programmierbar sein. Die Softwarekomponente besteht nur aus der Software-Laufzeitumgebung und ihrem Betriebssystem. Die Hardware umfasst verschiedene Aspekte wie Signalprozessoren, Speicher, Kommunikationsmodule usw. Zu den Anwendungen für eingebettete System gehören Industrie- und Landmaschinen, Unterhaltungselektronik, Transportmittel, medizinische Geräte, Kameras. Haushaltsgeräte, Verkaufsautomaten und Spielzeug sowie mobile Geräte.

Die Hardware eingebetteter Systeme kann auf Embedded Mikroprozessoren oder Embedded Mikrocontrollern basieren. Die Embedded Mikroprozessor Unit (EMPU) ist ein Mikroprozessor, der die Standard-CPU eines Universal-Computers auf einer speziell entwickelten Leiterplatte unterbringt und nur die Funktionen der Hauptplatine für die eingebettete Anwendung beibehält, um ein eingebettetes System zu bilden. Im Vergleich zu einem Universal-Computer sind die Systemgröße und der Stromverbrauch deutlich geringer, während der Betriebstemperaturbereich, die Unempfindlichkeit gegenüber elektromagnetischen Störungen und die Systemzuverlässigkeit verbessert sind. Die eingebettete Mikrocontroller verwendet ein Mikroprozessor als Kern, der Chip intern mit einer bestimmten Kapazität von Speicher (ROM/EPROM/ROM), I/O.Schnittstelle (serielle Schnittstelle, parallele Schnittstelle), Timer/Zähler, Watchdog, Pulsweitenmodulation Ausgang, A/D-Wandler, D/A-Wandler, Bus, Bus-Logik, etc. integriert.

2.2.1 Arduino

Das Arduino-Projekt ist ein 2006 gestartetes Projekt aus Italien. Arduino ist eine Zusammenstellung aus verschiedenen Boards (Mikrocontroller) und einer IDE. Das Gesamt Projekt ist Open-Source. Die Arduino-Boards sind auf einer einfachen Ausgabe-/Eingabe-Schnittstellenkarte aufgebaut und verfügt über eine Verarbeitungs/Verdrahtungs-Entwicklung mit Java, C++ Sprache ähnlichen.

Der Vorteil des Arduino ist, dass er über eine Funktionsbibliothek verfügt, die von fast allen Mikrocontrollern übertroffen wird. Und die Vielfalt der Sensoren ist sehr vielseitig und kann mit fast jedem Arduino-Board kompatibel sein. Darüber hinaus steht es mehrere Entwicklungsumgebungen (IDE) für Elektronik-Entwickler zur Verfügung, wie *Arduino IDE* ¹ und *Plattform IO* ².

Die meisten Arduino-Boards werden mit der C/C++ Sprache programmiert, die Struktur des Arduino-Programms unterscheidet sich jedoch wiederum von der eines traditionellen C/C++ Programms. Es gibt keine *main()*-Funktion im Arduino Programm. Es ist nicht so, dass Arduino-Programme keine *main()*-Funktion haben, sondern dass die Definition der *main()*-Funktion in der Datei der Arduino-Kernbibliothek versteckt ist. Bei der Entwicklung mit Arduino wird nicht direkt auf die *main()*-Funktion gearbeitet, stattdessen mit den Funktionen *setup()* und *loop()*. Das Programm in der Funktion *setup()* wird ausgeführt, nachdem der Arduino Board eingeschaltete oder zurückgesetzt wurde, und wird nur einmal ausgeführt. In *setup()* erfolgt in der Regel die Initialisierung des Arduino-Programms z.B. die Initialisierung von Variablen, die Initialisierung der seriellen Schnittstelle, die Konfiguration des E/A-Status, die Aktivierung der Softwarebibliothek usw. Sobald das Programm in *setup()* ausgeführt wurde, führt der Arduino das Programm in der Funktion *loop()* aus. Dabei handelt es sich um eine unendliche Schleife, sodass der Arduino kontinuierlich arbeiten kann. Hier werden in der Regel die Hauptfunktionen des Programms, wie die Ansteuerung verschiedener Module und die Datenerfassung, abgeschlossen.

Der folgende Abschnitt wird das in diesem Projekt (SOAAP) verwendete Arduino-Board (Arduino Nano 33 BLE) beschrieben.

¹ *Arduino IDE* [8] ist eine eigene von Arduino mitgebrachte integrierte Entwicklungsumgebung, die auf Wiring IDE basiert. Die bringt einen Code-Editor mit und bindet *gcc* als Compiler ein. Zusätzlich werden die *avr-gcc-Library* und weitere Arduino-Bibliotheken eingebunden, die die Programmierung in C und C++ stark vereinfachen

² *Plattform IO* ist auch ein Open-Source-IDE, die in Visual Studio untergebaut ist. *Plattform IO* hat eine funktionierende Verwaltung von Abhängigkeiten (inklusive Versionsverwaltung). Was besonders ist, dass man für jeden Projektordner eine Konfigurationsdatei anlegt, die alle nötigen Einstellungen für Hardware enthält. Damit ist es ein Leichtes, ein Projekt weiterzugeben oder Projekt anderer Entwickler zu verwenden.

2.2.2 Esp8266

Der *Esp8266* [9] ist ein kostengünstiger WiFi-Mikrochip mit integrierter TCP/IP-Netzwerksoftware und Mikrocontroller-Funktionen, der von Espressif Systems in Shanghai, China, hergestellt wird. Er kann als Mikrocontroller-Chip verwendet werden. Der *ESP32* ist eine Serie von kostengünstigen, stromsparenden System Single-Chip-Mikrocontrollern, die auf den *Esp8266* basiert.

Um ein WiFi-Modul in das SOAAP-System einzubinden, wird das Esp8266-Modul verwendet. Und die Kommunikation zwischen Arduino Nano 33 BLE und Esp8266 wird über die serielle Schnittstelle (UART) realisieren. Das ausgewählte Board ist ein *Wemos D1 mini pro*.

2.3 Kommunikationstechnologien

Bei der Entwicklung von intelligenten Geräten ist die Kommunikation zwischen verschiedenen Geräten ein sehr wichtiger Aspekt. Diese intelligenten Geräte kommunizieren miteinander, tauschen Daten aus und führen eine Reihe von Aufgaben aus, ohne dass ein Mensch eingreifen muss. Die Kommunikation kann verdrahtet erfolgen, die wird meist zwischen verschiedenen Komponenten innerhalb des Mikrocontrollers oder zwischen Sensoren und Mikrocontrollern verwendet, z.b. wie über SPI, I2C oder UART usw. Die Kommunikation kann auch drahtlos erfolgen, z.b. über das Internet oder Bluetooth usw. Der technische Entwickler kann die Hardware entwerfen und eine geeignete Kommunikationslösung entsprechend den tatsächlichen Anforderungen auswählen. Wie im SOAAP-Projekt dürften die Sensoren nicht die Bewegung des Künstlers (Spieler) einschränken, wenn sie am Körper getragen werden. Daher ist in dem Fall die drahtlose Kommunikation die einzige Option, z.b. ist Low Energie Bluetooth die beste Wahl.

In dieser Arbeit werden sich auf WiFi-Kommunikation und UART-Kommunikation bezogen, andere Kommunikationstechnologien werden nicht beschrieben.

2.3.1 OSI-Modell und TCP/IP-Protokollstruktur

Das OSI-Modell (Open Systems Interconnection Modell) [10] ist ein konzeptionelles Modell, das von der Internationalen Organisation für Normung entwickelt wurde, um Kommunikationssysteme mit Standardprotokollen zu unterstützen. Vereinfacht ausgedrückt, bietet OSI einen Standard, der sicherstellt, dass verschiedene Computersysteme miteinander kommunizieren können. Das OSI-Modell ist auch die gebräuchliche Sprache für Computernetzwerke und ist in sieben Abstraktionsschichten unterteilt ist (sortiert von oben nach unten: *Application Layer*, *Presentation Layer*, *Session Layer*, *Transport Layer*, *Network Layer*, *Data Link Layer*, *Physical Layer*), wobei jede Schicht für die Bearbeitung spezifischer Aufgaben und die Kommunikation mit den oberen und unteren Schichten zuständig ist.

Das TCP/IP-Modell ist ebenfalls ein geschichtetes Referenzmodell, aber es ist ein Vier-Schichten-Modell (sortiert von oben nach unten *Application Layer*, *Transport Layer*, *Internet Layer*, *Network Access Layer*). Ein anderer Name dafür ist *Internet Protocol Suite*. Sie ist allgemein als TCP/IP bekannt, weil die grundlegenden Protokolle TCP und IP sind. Jedoch werden in diesem Modell nicht nur diese beiden Protokolle verwendet.

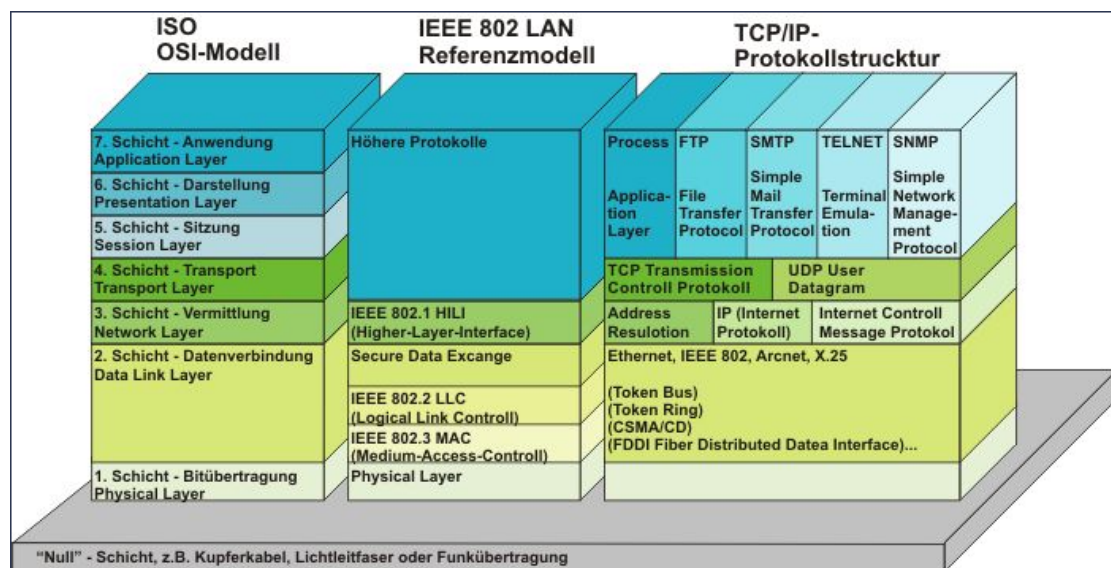


ABBILDUNG 2.1: OSI-Modell und TCP/IP-Modell

In Abbildung 2.1 zeigt die Relevanz der beiden Modelle und sieht man ein Referenzmodell (IEEE 802 LAN) dazwischen. Der Unterschied zwischen den beiden Modellen besteht

darin, dass OSI-Modell ein theoretisches Modell ist, dessen Zweck es ist, dass alle Unternehmen diese Spezifikation verwenden, um das Netzwerk zu kontrollieren, TCP/IP-Protokollstruktur ist ein verallgemeinertes Modell, das auf realen Situation basiert und sich auf die Struktur des OSI-Modells verweist. Die strukturellen Unterschiede [11] sind, dass die obere drei Schichten vom OSI/Model (*Application Layer*, *Presentation Layer* und *Session Layer*) in der TCP/IP-Protokollstruktur zu einer Schicht *APPLICATION* zusammengefasst (siehe Abbildung 2.1 den blauen Teil der TCP/IP-Protokollstruktur). Beziehungsweise werden die unteren zwei Schichten (*Data Link Layer* und *Physical Layer*) im OSI-Model zu einer Schicht *NETWORK INTERFACE* kombiniert.

Die Bedeutung und die Aufgaben der einzelnen Schichten vom TCP/IP-Modell sind :

Die *Anwendungsschicht* (*APPLICATION*) der TCP/IP-Protokollstruktur bietet Anwendungen die Möglichkeit, auf Dienste der anderen Ebenen zuzugreifen, und definiert die Protokolle, die Anwendungen zum Datenaustausch verwenden. Die bekanntesten Protokolle der Anwendungsschicht sind HTTP, FTP, SMTP, Telnet, DNS, SNMP und Routing Information Protocol (RIP).

Die *Transportschicht* (*TRANSPORT*) ist verantwortlich für die Bereitstellung von Session- und Datagramm-Kommunikationsdiensten für die Anwendungsschicht. Die Kernprotokolle dieser Schicht sind TCP und UDP. TCP bietet einen One-to-One-, verbindungsorientierten und zuverlässigen Kommunikationsdienst. Es ist verantwortlich für die Sequenzierung und Bestätigung der gesendeten Pakete und die Wiederherstellung von Paketen, die bei der Übertragung verloren gegangen sind. UDP bietet einen One-to-One- oder One-to-Many-, verbindungslose und unberechenbare Kommunikationsdienste. UDP wird typischerweise verwendet, wenn die zu übertragende Datenmenge eher klein ist (z.B. wenn die Daten in ein einziges Paket passen würden).

Die *Internetschicht* (*INTERNET*) ist für die Host-Adressierung, Paketierung und Routing-Funktionen verantwortlich. Die Kernprotokolle der Internet-Protokollschicht sind IP, Address Resolution Protocol (ARP), Internet Control Message Protocol (ICMP) und Internet Group Management Protocol (IGMP). IP ist ein routingfähiges Protokoll, das für die IP-Adressierung, das Routing und die Fragmentierung und Wiederausammenfügung von Paketen verantwortlich ist. Das ARP ist verantwortlich für die Ermittlung der Adresse der Netzzugangsschicht, wie z.B. einer Hardware-Adresse, die mit einem bestimmten

Internet-Schichtzugang verbunden ist. Das ICMP ist für die Bereitstellung von Diagnosefunktionen und die Meldung von Fehlern aufgrund der erfolglosen Zustellung von IP-Paketen verantwortlich. Die IGMP ist für das Management von IP-Multicast-Gruppen verantwortlich. In dieser Schicht fügt die IP den Paketen einen Header hinzu, der als IP-Adresse bezeichnet wird. Nun gibt es sowohl eine IPv4 (32-Bit)-Adresse als auch eine IPv6 (128-Bit)-Adresse.

Die *Netzwerkzugriffsschicht* oder *Verbindungsschicht* (*NETWORK INTERFACE*) ist dafür verantwortlich, die TCP/IP-Pakete auf dem Netzwerkmedium zu platzieren und entsprechende Pakete vom Netzwerkmedium zu empfangen. TCP/IP ist so konzipiert, dass es unabhängig von der Netzwerkzugriffsmethode, dem Frame-Format und dem Medium arbeiten kann. Mit anderen Worten: es ist unabhängig von einer bestimmten Netzwerktechnologie. Auf diese Weise kann TCP/IP verwendet werden, um verschiedene Netzwerktypen wie Ethernet, Token Ring und Asynchronous Transfer Mode (ATM) zu verbinden.

2.3.1.1 Socket

Der Socket ermöglicht die Kommunikation zwischen zwei verschiedenen Prozessen auf demselben oder verschiedenen Rechnern. Der verstecken die komplexe Familie der TCP/IP-Protokolle hinter einer Socket-Schnittstelle [12]. Es ist wie die Kommunikation mit anderen Computern über Standard-Unix-Dateideskriptoren. Unter Unix wird jede E/A-Aktion durch Schreiben oder Lesen eines Dateideskriptors ausgeführt. Ein Socket verhält sich tatsächlich ähnlich wie ein Low-Level-Dateideskriptor. Das liegt daran, dass Befehle wie *read()* und *write()* mit Sockets auf die gleiche Weise funktionieren wie mit Dateien und Pipes. Sockets verstecken die komplexe Familie der TCP/IP-Protokolle hinter einer Socket-Schnittstelle, sodass Entwickler einfach die Datenkommunikation zwischen zwei Geräten im Netzwerk über eine bestimmte Portnummer oder IP-Adresse implementieren können.

Es gibt zwei Arten von Sockets, die üblicherweise verwendet werden, nämlich *Stream Socket* und *Datagram Socket*, und Prozesse kommunizieren nur zwischen Sockets mit demselben Arten.

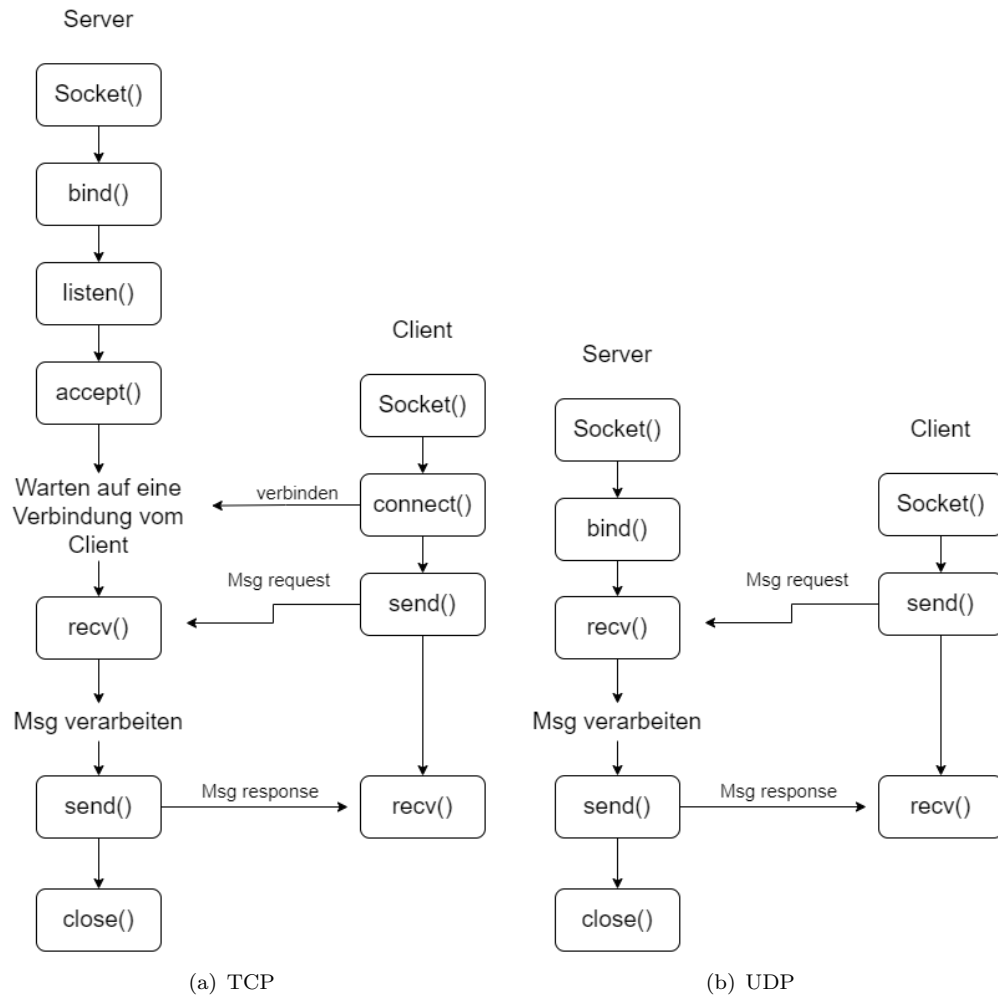


ABBILDUNG 2.2: Datenaustausch mit der Socket-Funktion unter TCP und UDP [1]

Der *Stream Socket* verwendet *TCP* (*Transmission Control Protocol*) für die Datenübertragung. Auf diese Weise wird die Datenübertragung in einer Netzumgebung gewährleistet. Die Abbildung 2.2(a) zeigt, wie der *Stream Socket* programmiert wird. Das gesamte Programm lässt sich auf die folgenden Funktionen reduzieren werden, `socket()`, `bind()`, `listen()`, `accept()`, `recv()`, `send()` und `close()`. Als Erstes wird eine Verbindung zwischen dem Client und dem Server hergestellt. Die Verbindungsanfrage wird vom Client gestellt, über *three Handshake Process* wird die Verbindung erfolgreich hergestellt. Die IP-Adresse und der Port des Servers müssen dem Client während dieses Vorgangs bekannt sein. Da das TCP-Protokoll einen Rückmeldungs Aufbau (Im Falle eines Paketverlusts oder -fehlers wird das richtige Paket wiederholt gesendet, bis das Ziel es empfängt) enthält, wird die Sicherheit und die Zuverlässigkeit des Datenaustausches garantiert. Allerdings nehmen dieser Vorgang einige Zeit in Anspruch.

Der *Datagram Socket* verwendet *UDP* (*User Datagram Protocol*) für die Datenübertragung.

UDP ist ein verbindungsloses Protokoll. Der Programmierungsprozess vom *Datagram Socket* sieht man in der Abbildung 2.2(b), bei diesem Vorgang beginnt der Datenaustausch ohne die zuverlässige Verbindung zwischen dem Server und dem Client. Nach dem Senden des Datenpakets ist es irrelevant, ob das Zielgerät die Daten empfängt, sodass es nicht zuverlässig ist, aber im Gegensatz dazu bietet die höhere Übertragungsgeschwindigkeit.

2.3.2 WiFi-Kommunikation

1997 wurde die erste Version des drahtlosen Protokolls, *IEEE 802.11*, veröffentlicht. *IEEE 802.11* ist eine Reihe von Standards, die für die drahtlose LAN-Kommunikation definiert wurden (in Übereinstimmung mit IEEE, Institute of Electrical and Electronics Engineers).

WLAN (Wireless Local Area Networks) ist ein drahtloses lokales Netzwerk, das auf der *IEEE 802.11*-Normenreihe basiert und als Übertragungsmedium hochfrequente Funkfrequenzen (z. B. drahtlose elektromagnetische Wellen im 2,4-GHz- oder 5-GHz-Band) verwendet. Es gibt viele Kommunikationstechnologien im WLAN-Netzsystem, WiFi ist eine der am weitesten verbreiteten in der Gesellschaft [13].

2.3.2.1 Access Point

Access Point (kurz *AP*) [14] ist ein Zugangspunkt. Es handelt sich um ein Gerät in einem drahtlosen Netzwerk, das das drahtlose Netzwerk erzeugt und einem Benutzer (Geräte, die drahtlose Netzwerke unterstützen) den Zugang zu diesem Netzwerk ermöglicht (siehe Abbildung 2.3). *Access Point* werden verwendet, um die Funkabdeckung eines bestehenden Netzwerks zu erweitern und die Anzahl der Benutzer, die sich mit dem Netzwerk verbinden können, zu erhöhen.

Eine weitere Funktion eines AP ist das **Relaying**. Das drahtlose Signal wird einmal durch den AP verstärkt, sodass drahtlose Geräte in größerer Entfernung ein stärkeres Signal empfangen können. Der AP ermöglicht es auch, zwei drahtgebundene lokale Netzwerke zu überbrücken. Durch die Installation eines AP in zwei getrennten kabelgebundenen lokalen Netzwerken können zwei Geräte in verschiedenen Netzwerken Daten untereinander übertragen.

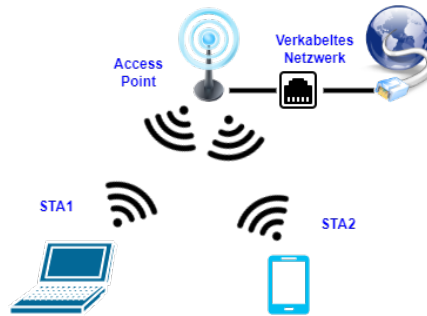


ABBILDUNG 2.3: Access Point

2.4 Serielle Schnittstelle - UART

UART steht für *Universal Asynchron Receiver and Transmitter*. Für die Kommunikation werden nur zwei Datenleitungen benötigt. Eine zum Senden der Daten (*Tx* - Transmit) und eine zum Empfangen der Daten (*Rx* - Receive).

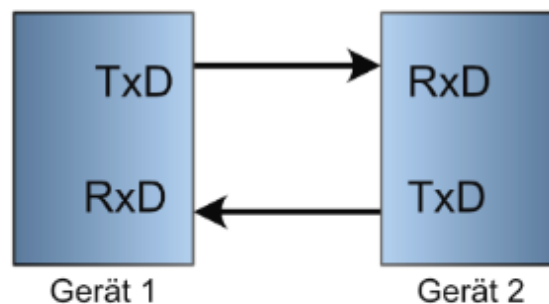


ABBILDUNG 2.4: Datenaustausch der UART Kommunikation [2]

UART verwendet keinen Takt, um das Signal zu synchronisieren, sondern Start- und Stopbits. Wenn der Empfänger das Startbit empfängt, beginnt er mit dem Lesen der Daten, bis er das Stoppbit empfängt. Eines der wichtigsten Konzepte ist die *BAUD-Rate*, die die Geschwindigkeit festlegt, mit der die Daten gesendet werden. Die am zweithäufigsten verwendeten *BAUD-Raten* sind 9600 und 115200, was bedeutet, dass bei 9600/s jedes Bit in einem Abstand von 1042 ms gesendet wird, oder bei 115200/s jedes Bit in einem Abstand von 86.806 μ s gesendet wird.

Das Telegramm sieht folgendermaßen aus:

Mit dem Paritätsbit kann eine grobe Kontrolle durchgeführt werden, ob alle Daten korrekt angekommen sind. Am Ende der Übertragung werden ein oder zwei Stopbits gesetzt.

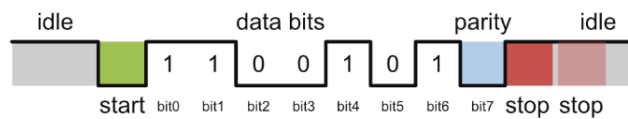


ABBILDUNG 2.5: Telegramm der UART Kommunikation [2]

2.5 MIDI

MIDI steht für Musical Instrument Digital Interface, es ist eine Sprache oder ein Protokoll, die es Computern, Musikinstrumenten und anderer Hardware erlaubt, miteinander zu kommunizieren. Es gibt viel über MIDI zu sagen, aber diesem Abschnitt geht es nur um die Übertragung von Midi auf physikalischer Ebene.

2.5.1 MIDI Verbindungen

Heute gibt es drahtgebundene und drahtlose MIDI-Übertragungstechniken. Da viele Geräte die drahtlose MIDI-Übertragung noch nicht unterstützen, ist die drahtgebundene Übertragungstechnik weiter verbreitet. Für die meisten MIDI-Verbindungen werden 5-polige DIN-Kabel verwendet (DIN steht für das Deutsche Institut für Normung, eine deutsche Normungsorganisation; das ursprüngliche MIDI-Protokoll sah eine 5-polige DIN-Übertragung vor). Während für die meisten MIDI-Verbindungen ausschließlich MIDI-Kabel verwendet werden, gibt es auch Setups, die je nach Ausstattung oder Art des MIDI-Interfaces nur eine USB-zu-USB- oder USB-zu-MIDI-Verbindung benötigen.

5-pin DIN-Kabel und Anschluss

Der Aufbau des MIDI-Kabels ist in der Abbildung 2.6 dargestellt. Der äußere Teil des Kabels ist von einem Schutzgehäuse umgeben, das geerdet ist, um die Einstreuung externer Signale in das Kabel zu reduzieren. Im Kabelinneren befinden sich zwei miteinander verdrehte Adern.

Auf beiden Seiten befindet sich die zwei DIN-Anschlüsse. Er hat jeweils fünf Stifte, von denen nur drei verwendet werden. Pins Information sieht man im Tabelle unten:

5-pin DIN-Buchsen

Es gibt zwei Arten von DIN-Buchsen, IN und OUT. In einem DIN-Kabel ist die Signalübertragung unidirektional, es werden also zwei Kabel benötigt, um die Datenübertragung

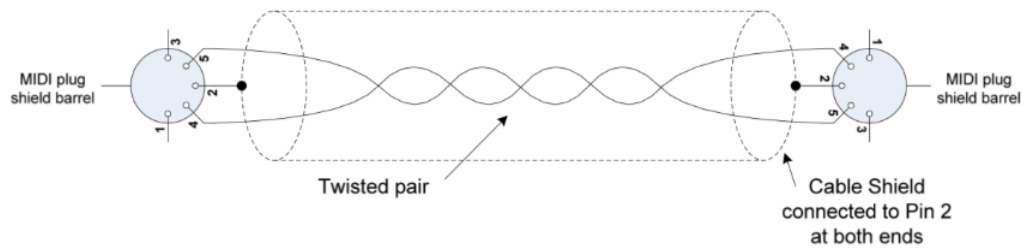


ABBILDUNG 2.6: Aufbau des 5-pin DIN-Kabel

PINName	Verwendungszweck
1	NC
2	Erdung des Schutzgehäuses
3	NC
4	Spannungsreferenzleitung
5	Daten Leitung

TABELLE 2.1: Pin-List des 5-pin DIN-Anschluss

in beide Richtungen zu gewährleisten, und die Schaltungen sind für die IN- und OUT-Buchsen unterschiedlich aufgebaut, siehe Bild 2.7 (Die THRU-Buchse ist die Weiterleitung des an der IN-Buchse empfangenen MIDI-Originalsignals, auf die hier nicht näher eingegangen wird).

An die MIDI-OUT-Buchse werden 3 Leitungen angeschlossen, PIN 4, 5 und PIN 2, der für die Erdung des Schutzgehäuses sorgt. An PIN 5 werden die Daten vom Mikroprozessor oder anderen digitalen Prozessoren als UART übertragen, die Signale werden durch die beiden im Bild dargestellten Verstärker verstärkt, sodass sie den Widerstand an der Schnittstelle passieren und über die Leitung übertragen werden können. An PIN 4 befindet sich eine 5-Volt-Versorgung und ein 220-Ohm-Widerstand. Bei der Buchse MIDI-IN werden 2 Leitungen verwendet, eine auf PIN 5 sorgt für die Übertragung der MIDI-Daten, eine auf PIN 4 sorgt für die Spannungsreferenzleitung und dort ist noch ein 220 Ohm Widerstand, ebenso bei Midi-OUT. Zwischen der Buchse und den internen Schaltkreisen befindet sich noch ein Optoisolator, er isoliert die elektronischen Signale von außen vollständig von den elektronischen Signalen der internen Schaltkreise, außerdem gibt es an der Buchse des MIDI-INS keinen PIN, der mit dem Schutzgehäuse des Kabels verbunden werden könnte, dadurch ist der äußere Ring der internen Schaltkreise frei von Störungen durch Signalrauschen der externen Schaltkreise.

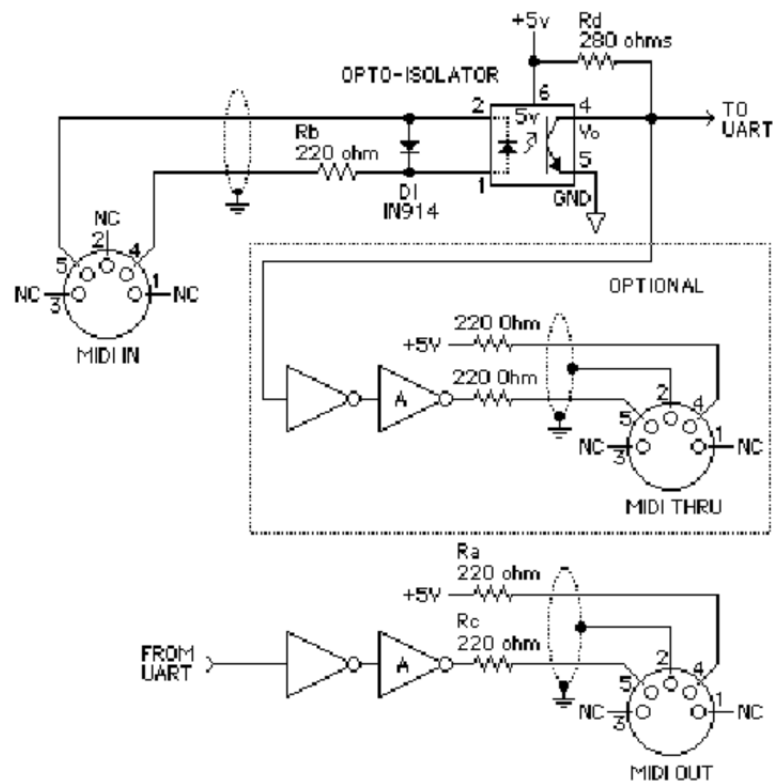


ABBILDUNG 2.7: Schaltplan der DIN-Buchsen IN/THRU und OUT

2.5.2 MIDI-Interface

Ein MIDI-Interface ist ein Mittel zur Übertragung von Noten- und Klanginformationen zwischen verschiedenen MIDI-Geräten und Computern. Auf der einen Seite des MIDI-Interfaces befindet sich ein USB-Anschluss für den Anschluss an einen Computer, auf der anderen Seite befinden sich ein oder mehrere MIDI IN- und OUT-Anschlüsse (meist 5-polige DIN-Buchsen oder Stecker), die mit MIDI-Instrumenten, z.B. MIDI-Keyboard oder anderen MIDI-Geräten, verbunden werden können. Das MIDI-Interface kann MIDI-Signale von mehreren MIDI-Instrumenten gleichzeitig empfangen oder senden. Die empfangenen Signale werden über das USB-Kabel geordnet an den Computer übertragen.

Das in diesem Projekt verwendete MIDI-Interface ist das MIDI2x2 der Firma *Midi-PLUS*. Es bietet eine USB-Schnittstelle, zwei MIDI-OUT-Schnittstellen und zwei MIDI-IN-Schnittstellen.



ABBILDUNG 2.8: MIDI-Interface Produkte



ABBILDUNG 2.9: MIDI-Interface MIDI2x2 von MidiPLUS

2.6 Eulerwinkel

Die Eulerwinkel ist eine Methode zur Beschreibung von Drehungen im dreidimensionalen Raum. Sie bestehen aus drei Winkeln, die oft als Roll-, Pitch- und Yawwinkel bezeichnet werden. Der Rollwinkel beschreibt die Drehung um die x-Achse, der Pitchwinkel um die y-Achse und der Yawwinkel um die z-Achse. Zusammen bilden diese Winkel eine eindeutige Darstellung einer beliebigen Drehung im Raum. Die Verwendung von Euler-Winkeln ist in vielen Anwendungen wie Robotik, Luftfahrt und Navigation weit verbreitet.

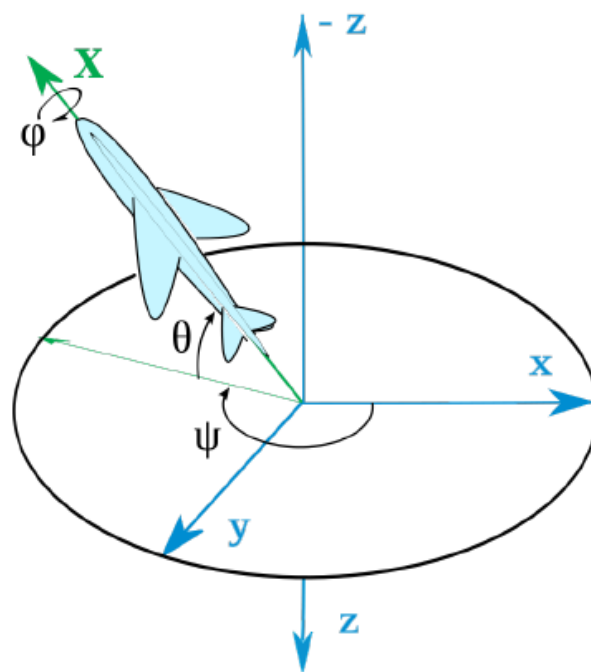


ABBILDUNG 2.10: Roll-, Pitch- und Yawwinkel vom Eulerwinkel

Kapitel 3

Technisches Umfeld und theoretischer Teil der Arbeit

3.1 Android Programmierung

3.1.1 JAVA

Java [15] ist eine im Jahr 1995 erschienene, objektorientierte Programmiersprache, die früher von der Firma *Sun Microsystems*, heute von *Oracle* entwickelt wird. Die Programmiersprache ist Teil der *Java*-Technologie, die aus dem *Java*-Entwicklungswerkzeug (Java Development Kit (JDK)) zur Erstellung von *Java*-Programmen und der, *Java*-Laufzeitumgebung (Java Runtime Environment (JRE)), zu deren Ausführung besteht. Das JDK ist das meistgenutzte SDK für *Java*-Entwicklungen, die JRE besteht aus der virtuellen Maschine (Java virtual Machine (JVM)) und den zur Ausführung benötigten Bibliotheken. Aufgrund der Tatsache, dass *Java* innerhalb einer virtuellen Maschine läuft, ist es besonders plattformunabhängig. *Java* läuft theoretisch auf jeder Rechnerarchitektur, solange dort die nötige JRE installiert wurde.

3.1.2 Thread Programmierung

Wenn eine Anwendung gestartet wird, erstellt das System einen Ausführungsthread für die Anwendung, der "Main" genannt wird. Dieser Thread ist sehr wichtig, da er für die Verteilung von Ereignissen und die Darstellung der Benutzeroberfläche zuständig ist.

Alle UI-Komponenten (Aktivitäten, Dienste usw.) und ihr ausgeführter Code laufen um selben Prozess und werden standardmäßig im UI-Thread (*Main-Thread*) instanziiert. Operationen im UI-Thread, die über längere Zeiträume ausgeführt werden, wie z.b. Netzwerkzugriffe oder Datenbankabfragen, sind nicht zulässig und blockieren die UI-Antwort für die gesamte Anwendung. Wenn der UI-Thread blockiert ist, können keine Ereignisse gesendet werden, auch keine Zeichenereignisse. Aus Sicht des Benutzers erscheint die Anwendung dann starr.

Aufgrund der oben beschriebenen großen Probleme beim Blockieren von UI-Threads sollte jede Android-Anwendung Hintergrund-Threads verwenden, um alle langlaufenden Aufgaben auszuführen. Im Android Framework gibt es jedoch verschiedene Abstraktionen für die Thread-Verwaltung.

AsyncTask, Handler Thread und ThreadPoolExecutor [\[16\]](#)[\[17\]](#)[\[18\]](#)

AsyncTask ermöglicht die einfache Ausführung kurzer sequentieller Aufgaben innerhalb eines Aktivitätskontextes. *AsyncTask* wird häufig verwendet, um eine Hintergrundaufgabe auszuführen, den Fortschritt zu melden und dann den UI-Thread mit den Ergebnissen zu aktualisieren. Die Verwendung der parallelen Ausführung von Tasks mit *AsyncTask* hat sich jedoch weiterentwickelt und häufig geändert und kann daher etwas fehleranfällig sein. *AsyncTask* sollte für die Ausführung kurzer Operationen (maximal einige Sekunden) in sequentieller Reihenfolge verwendet werden.

HandlerThread ist eine praktische Klasse zum Starten eines neuen Verarbeitethreads, der Aufgaben der Reihe nach ausgeführt. Beispielsweise kann ein einzelner Hintergrund-Thread gestartet werden, um eine Schleife zu starten, die in der Lage ist, Code auszuführen oder Nachrichten in der Reihenfolge ihres Eintreffens zu verarbeiten. Der initialisiert einen *Looper* innerhalb eines Threads, um *Runnable*- oder *Message*-Objekte zu verarbeiten.

ThreadPoolExecutor ist eine hervorragende Möglichkeit, parallelisierte Aufgaben über mehrere verschiedene Threads innerhalb eines begrenzten Threadpools auszuführen. Er ermöglicht die gleichzeitige Ausführung von Aufträgen und behält die Kontrolle darüber, wie sie ausgeführt werden.

3.1.3 UI-Komponente

Die Benutzeroberfläche in dieser Arbeit wurde mit *Material Design* erstellt. *Material Design* ist eine Designsprache, die 2014 von *Google* entwickelt wurde, mit dem hochwertigen digitalen Erlebnisse für Android, iOS, Flutter und das Web erstellen können. *Material Design* baut auf den "Karten", die in *Google Now* eingeführt wurden, und verwendet mehr gitterbasierte Layouts, reaktionsfähige Animationen und Übergänge, Füllungen und Tiefeneffekte wie Licht und Schatten.

Um *Material Design* in Android zu verwenden, muss zuerst das *Google's Maven* Repository in die Datei *build.gradle* des Android-Projekts synchronisiert und die Bibliothek des *Material Designs* in die *dependencies* hinzugeführt werden. Zugang zum *Material Design* wird erst nach der oben genannten Schritte gewährt. Als Nächstes sollte das *APP-Theme* abgeändert werden, dass es von einem *Material Components-Theme* abgeleitet ist. Anschließend kann der Entwickler die verschiedenen erforderlichen Originale in der XML-Datei hinzufügen oder bearbeiten. Es gibt eine Reihe von Versionsbedingungen, die erfüllt sein müssen, damit das *Material Design* verwendet werden kann¹.

Häufig verwendete Komponenten sind [19]: Toolbars, Buttons, Drop-Dwons, Bottom/-Side Sheet, Backgrounds, Cards, Sliders usw.

3.1.4 Android Klasse für Stream Socket

Socket ist eine Klasse für Erstellung einer Stream-Socket-Netzwerkverbindung, die von der Android-Entwicklung im Package *java.net* zur Verfügung gestellt wird. Der *Public Constructors* ist *Socket(InetAddress address, int port)*, die hier erforderliche Argumenten sind die IP-Adresse und der Port des Zielgerätes. Zusätzlich zur IP-Adresse kann auch der *Hostname* als *String* angegeben werden.

Folgt ein Beispiel für den Code zur Erstellung eines clientseitigen Sockets [12].

```
1 Socket client;  
2  
3 try{  
4     client = new Socket("192.168.0.1", 4321);
```

¹Weitere Informationen findet man in der offiziellen Dokumentation *Getting Started* vom *Material Design* <https://m2.material.io/develop/android/docs/getting-started>

```

5   out = new PrintWriter(client.getOutputStream(),true);
6   in = new BufferedReader(new InputStreamReader(client.
      getInputStream()));
7
8 } catch (UnknownHostException e) {
9     System.out.println("Unknown Ip Adresse");
10
11 } catch (IOException e) {
12     System.out.println("No I/O");
13 }

```

Um Informationen aus einem Socket zu lesen und zu schreiben, kann Klassen *InputStream* und *OutputStream*. Die *Public Constructors* sind *InputStream()* und *OutputStream()* mit keinen Argumenten. Die grundlegenden Methoden davon sieht man in die Tabellen 3.1 und 3.2.

InputStream

Methode	Erklärung
int available()	Eine Schätzwert für die Anzahl der Bytes wird zurückgegeben, die aus diesem Inputstream gelesen (oder ignoriert) werden können
void close()	Inputstream schließen
int read(byte[] b)	Eine bestimmte Anzahl von Bytes aus dem Inputstream wird gelesen und im Puffer-Array <i>b</i> gespeichert

TABELLE 3.1: Methoden der Klasse *InputStream*

OutputStream

Methode	Erklärung
void close()	Inputstream schließen
void flush()	Der Outputstream wird geleert und die Ausgabe aller gepufferten Ausgabebytes erzwungen
void write(byte[] b)	<i>b</i> lange Bytes aus dem angegebenen Byte-Array in diesen Outputstream wird geschrieben

TABELLE 3.2: Methoden der Klasse *OutputStream*

3.2 Mikrocontroller

3.2.1 Arduino Nano 33 BLE

Der Arduino Nano ist eine Platine im Arduino-Projekt. Er ist der kleinste 8-Bit-Mikrocontroller von Arduino, verwendet einen ATmega328-Mikrocontroller mit AVR-Architektur.

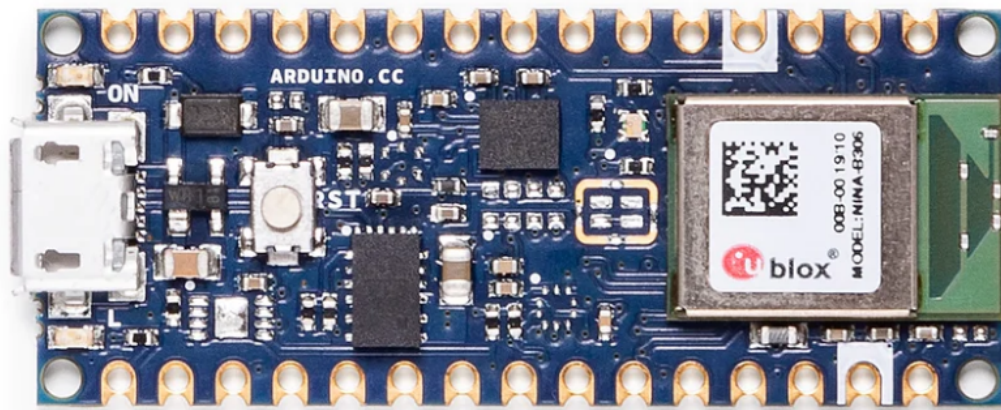


ABBILDUNG 3.1: Vorderansicht des Arduino Nano 33 BLE [3]

In dieser Arbeit wird der Arduino Nano 33 BLE (siehe Bild 3.1) als Hauptelement verwendet. Der Arduino Nano 33 BLE ist ein weiterentwickeltes Board vom vorherigen Arduino Nano. Er ist ein kompaktes Board, das um das NINA B306 Modul herum aufgebaut ist, das auf dem Nordic nRF52480 (einer 32-Bit ARM®-M4 CPU mit 64 Mhz) basiert. Das Gerät verfügt noch über eine Inertial Messeinheit mit 9 Achsen (IMU) und verbraucht weniger Strom als andere Arduino Boards mit derselben Größe. Der Arduino Nano 33 BLE ist dank des Multiprotokoll-Funks BT 5.0 ideal für Automatisierungsprojekte geeignet.

Das SOAAP-Projekt erforderte drahtlose Kommunikation zur Geräteunterstützung, IMU-Sensoren zur Messung von Bewegungsdaten und einen stromsparenden Mikrocontroller auf der Wearable-Seite. Der *Arduino Nano 33 BLE* verfügt über alle Features, die für SOAAP-Projekt benötigt werden, sodass er die perfekte Wahl für dieses Projekt ist.

In der Tabelle 3.3 sind die technischen Features des Arduino Nano 33 BLE zu sehen.

Mikrocontroller	nRF52480
Betriebsspannung	3,3 V
Eingangsspannung (begrenzt)	21 V
DC-Strom pro E/A-Pin	15 mA
Taktgeschwindigkeit	64 Mhz
CPU Flash-Speicher	1 MB (nRF52480)
SRAM	256 KB (nRF52480)
EEPROM	keine
Digitaler Eingang/Ausgang Pins	14
PWM Pins	alle digitalen Pins
UART	1
SPI	1
I2C	1
Analoge Ausgang Pin	8(ADC 12 bit 200 ksamples)
Analoge Eingang Pin	nur über PWM (kein DAC)
Externe Unterbrechungen	alle digitalen Pins
USB	Nativ im nRF52480-Prozessor
Länge	45 mm
Breite	18 mm
Gewicht	5 gr (mit Headers)

TABELLE 3.3: Arduino Nano 33 BLE Tech specs [3]

3.2.2 Wemos D1 mini pro

Wemos D1 mini pro ist ein Mini-WiFi-Board mit 16 MB Flash, externem Antennenanschluss und eingebauter Keramikantenne basierend auf dem *esp8266ex*. In Abbildung 3.2 sieht man eine Vorderansicht des Boards.

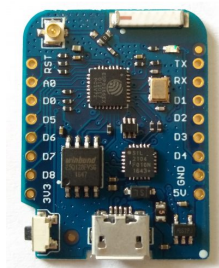


ABBILDUNG 3.2: Vorderansicht des Wemos D1 mini pro [4]

In der Tabelle 3.4 sind die technischen Features des Arduino Nano 33 BLE zu sehen.

Mikrocontroller	esp8266ex
Betriebsspannung	3,3 V
Taktgeschwindigkeit	80/160MHz
Flash-Speicher	16 MB
Digitaler Eingang/Ausgang Pins	11
Analoge Eingang Pins	1 (3,2V Max)
Dimension	34,4x25,6x4mm
Gewicht	2,5 gr

TABELLE 3.4: Wemos D1 mini pro Tech specs [4]

3.2.3 Esp8266 Arduino Core

Der *Esp8266 Arduino Core* ist eine Softwarebibliothek, die von der Arduino-Community entwickelt wurde, um den Esp8266/chip mit der Arduino-Entwicklungsumgebung programmieren zu können. Diese Bibliothek bietet eine Vielzahl von Funktionen, die Entwicklern ermöglichen, auf die GPIO-Pins des Esp8266 zuzugreifen, mit WiFi-Netzwerken zu kommunizieren und Daten aus dem Internet zu lesen oder zu senden. In *Esp8266 Arduino Core* gibts viele weitere Bibliotheken, wie z.b. *ESP8266WiFi*, *WiFiServer* und *WiFiClient*, die in dieser Arbeit im Einsatz sind.

ESP8266WiFi

Die ist eine Teilbibliothek des *ESP8266 Arduino Core* und ermöglicht die einfache Verbindung und Kommunikation mit WiFi-Netzwerken unter Verwendung des Esp8266-Chips. Die Namenskonvention und die Gesamtfunktionalität der Bibliothek sind ähnlich zu anderen von *Arduino* bereitgestellten Bibliotheken. Mit der Methode *WiFi.begin("network-name", "pass-to-network")* lässt sich die Netzwerkeinstellungen der WiFi-Bibliothek und zeigt den aktuellen Status an initialisieren. Die zwei Argumenten *network-name* und *pass-to-network* sind den Namen und das Kennwort des WiFi-Netzwerks, mit dem man sich verbinden möchte. Hier ist der Beispielcode, um das Esp8266 mit WiFi zu verbinden [20]:

```

14 #include <ESP8266WiFi.h>
15
16 void setup()
17 {

```

```
18 Serial.begin(115200);
19 Serial.println();
20
21 WiFi.begin("network-name", "pass-to-network");
22
23 Serial.print("Connecting");
24 while (WiFi.status() != WL_CONNECTED)
25 {
26     delay(500);
27     Serial.print(".");
28 }
29 Serial.println();
30
31 Serial.print("Connected, IP address: ");
32 Serial.println(WiFi.localIP());
33 }
34
35 void loop() {}
```

WiFi.status() meldet den Status der Esp8266-Verbindung mit dem Netzwerk und bedeutet nur dann eine erfolgreiche Verbindung, wenn der Rückgabewert *WL_CONNECTED* ist. Andernfalls verbleibt sie in der Verbindungsschleife.

Hier ist der Beispielcode, um einen Access Point zu erstellen [21]:

```
36 #include <ESP8266WiFi.h>
37
38 void setup()
39 {
40     Serial.begin(115200);
41     Serial.println();
42
43     Serial.print("Setting soft-AP ... ");
44     boolean result = WiFi.softAP("esp-softAP_01", "pass-to-soft-AP");
45     if(result == true)
46     {
47         Serial.println("Ready");
48     }
49     else
```

```
50     {
51         Serial.println("Failed!");
52     }
53 }
54
55 void loop()
56 {
57     Serial.printf("Stations connected = %d\n", WiFi.softAPgetStationNum
58         ());
59     delay(3000);
60 }
```

Mit der Methode *WiFi.softAP("esp-softAP_01", "pass-to-soft-A")* wird ein Access Point erstellt, die Argumenten *ESPsoftAP_01* und *pass-to-soft-AP* sind das *ssid* und *pass*, das andere Geräte benötigen, um sich mit dem Access Point zu verbinden.

WiFiServer und WiFiClient

Die Bibliothek *WiFiServer* ermöglicht die Einrichtung eines Webserver auf dem esp8266, ein notwendiger Schritte für die Netzwerkkommunikation. Die Bibliothek *WiFiClient* handelt sich um die Erstellung eines Clients. Diese Arbeit verwendet esp8266 als einen Server, aber einige Methoden vom *WiFiClient* kommen auch zum Einsatz. Unten sieht man eine vollständige Sketch [22]:

```
60 #include <ESP8266WiFi.h>
61 #include <WiFiServer.h>
62 #include <WiFiClient.h>
63
64 WiFiServer server(4210);
65
66 void setup()
67 {
68     // WiFi setup
69 }
70
71 void loop()
72 {
73     WiFiClient client = server.available();
```

```
74  if (client) {  
75      if(client.connected())  
76      {  
77          Serial.println("Client Connected");  
78      }  
79      while(client.connected()) {  
80          while(client.available()>0) {  
81              Serial.write(client.read());  
82          }  
83          while(Serial.available()>0)  
84          {  
85              server.write("hello from esp8266!");  
86          }  
87      }  
88      client.stop();  
89      Serial.println("Client disconnected");  
90  }  
91 }
```

Zuerst wird eine Instanz des Servers erstellt, im obigen Beispiel antwortet der Client auf Port 4210, der für das TCP-Protokoll vorgesehen ist. Der Instanz des Clients wird zu Beginn der Schleife *loop* erstellt, *server.available* gibt nur die Clients zurück, für die Daten verfügbar sind, und weist die der erzeugten Instanz des Clients zu. *client.connected()* gibt einen booleschen Wert zurück, wenn der *true* ist, ist der Client verbunden, andernfalls *false*². *client.available()* gibt die Anzahl der Bytes im Puffer zurück, die zum Lesen zur Verfügung stehen. *client.write()* liest das nächste empfangene Byte vom Server, mit dem der Client verbunden ist. *client.write()* befindet sich in einer While-Schleife und die Schleife endet, wenn alle Bytes im Puffer gelesen wurde, d.h. wenn der *client.available()* nicht mehr größer als 0 ist. Die gelesenen Bytes können in einer String oder Char-Array Variablen gespeichert werden. *server.write()* sendet Daten an alle mit dem Server verbundenen Clients.

²Doch eines ist zu beachten, dass ein Client als verbunden gilt, wenn die Verbindung geschlossen wurde, aber noch ungelesene Daten vorhanden sind

Kapitel 4

Über SOAAP

4.1 Was ist SOAAP

Die Technologie entwickelt sich immer ständig weiter und ist in alle Bereiche eingedrungen, wie Malerei, Musik, Gastronomie, Sport usw. Im Musikbereich, vom Grammophon bis zur digitalen Aufnahmetechnik, von der Schallplatte bis zum Online-Musik-Streaming, von traditionellen Instrumenten bis zu verschiedenen elektronischen Instrumenten und Instrumentensoftware, die Technologie hat die Musik für immer verändert. Die Menschen erforschen ständig den Weg, wie sie Musik aufführen können. SOAAP ist auch ein Fußabdruck auf diesen Weg.

SOAAP steht für Steuerung optischer und akustischer Ausgaben für Performance-Künstler, ist generell im Bereich von Mensch-Maschine-Interaktionen angesiedelt. Wir sind es gewohnt, zum Rhythmus der Musik zu tanzen. Aber vielleicht können wir diese Beziehung auf den Kopf stellen, Menschen verwandeln ihren Körper in ein Musikinstrument, durch Körper-bewegungen ersetzen das Spielen der Tasten auf dem Klavier, damit wundervolle Musik zu erstellen. Diese Idee wurde schon seit über einem Jahrzehnt entstanden und in die Praxis umgesetzt. Vertretene Produkte sind wie z. B. Hot Hand vom Firma Source

Audio¹ und SOMI-1 vom Firma Instruments of Things.² SOAAP wurde mit ähnlichen Zielen und Aufbaustruktur wie SOMI-1, besteht auch aus mehreren auf Basis des Arduino Boards entwickelten Sensoreinheiten und einem Empfänger, der Empfänger sammelt alle Messdaten vom IMU-Sensor an der Sensoreinheiten und damit werden MIDI-Signale über unterschiedliche Algorithmen erstellt. Schließlich werden die MIDI-Signale am Terminal (ein Computer oder ein Gerät, das das MIDI-Signal verarbeiten kann) verarbeitet und steuern die Musik oder die Beleuchtung.

4.2 Projekthintergrund SOAAP

Seit dem Frühjahr 2021 kooperiert ein Vierer-Team von Lehrenden der Fakultäten I und III an der Hochschule Hannover, um ein gemeinsames Forschungs- und Entwicklungsprojekt im Spannungsfeld von Elektrotechnik und Gestaltung zu erarbeiten. Hervorgegangen ist diese Zusammenarbeit aus einer fakultätsübergreifenden Kooperation, die im Rahmen des Kooperationsvertrages zwischen der Hörrigion Hannover und der Hochschule Hannover im Sommer 2019 ins Leben gerufen wurde. (*aus Prof. Hans-Jörg Kapp*)[5]

Das SOAAP Projekt entstand aus dieser Idee einer dieser umfassenden Forschungs- und Entwicklungsprojekt. Das Ziel davon ist zuallererst die Kommunikation über die Disziplingrenzen, etwas Gemeinsames zwischen mehr als einer Fakultät zu schaffen. Im Soaap Projekt beispielsweise arbeiten Professoren und Studenten aus der Hochschule Hannover Fakultät I und Fakultät III zusammen, wobei die Fakultät I für die technische Entwicklung und die Fakultät für die Erforschung von Aufführungsformen usw. zuständig sind. Einige der Konzepte des Projekts werden sich während des Austausches

¹Hot Hand richtet sich an Verbraucher, die Musiker sind. Der Hot Hand besteht aus zwei Teile, einen Sensor-Ring und einen Recevier. Der Ring (Hot Hand Motion Controller Ring) wandelt die Bewegungen der Hand mithilfe eines Beschleunigungsmessers in ein digitales Signal um, eine Technologie, die von Analog Devices entwickelt wurde. Der Ring in Kombination mit dem Motion Controlled Wah Filter Pedal erzeugt Effekte ähnlich dem Dunlop Cry Baby Wah, einem Pedal, das durch Spieler wie Jimmy Page und Jimi Hendrix berühmt wurde. Das Gerät bietet 11 verschiedene Wah-Filter, darunter klassisches Wah, Tiefpass, Bandpass, Multi-Peak und Volume-Swell, die eine breite Palette von Wah-Effekten erzeugen.

²SOMI-1 richtet sich an Tänzerinnen und Tänzer, die vielleicht keine musikalischen Kenntnisse haben, der besteht aus mehreren Sensor-Uhren, dass man die am Handgelenk oder am Knöchel tragen, und einen Hub. Der Sensor wird dann auf der X/Y/Z-Achse gekippt oder bewegt und sendet dann entsprechende Daten an die Software. Dadurch können z. B. Filter von Synthesizern per Handbewegung geöffnet oder geschlossen oder Drum Maschinen getriggert werden. Dafür stehen bisher unter anderem Kategorien wie Ambient, Natur oder Club zur Verfügung.

von beiden Seiten unterscheiden, aber genau das wird bei der Transdisziplinäre Kooperationen geschätzt, und dadurch wird auch die Perspektive der Studenten auf das Projekt bereichert. Darüber hinaus kommt noch zweites Ziel, dass im Rahmen des Projekts Studenten aus Fakultät I und der Gestaltung gemeinsam Sensoren bedienen lernen können und deren Nutzung als offenen kreativen Erkundungsprozess erleben können. Für die Gestaltungsstudierenden geht es darum, Erfahrungen in der Interaktion in vernetzten und technikbasierten Umgebungen zu gewinnen.

4.3 Konzept des SOAAPs

Die Arbeit bezieht sich auf das SOAAP-System vom Prof. Robert Patzke. Daher ist es notwendig, einen allgemeinen Überblick über die Struktur des Systems und einige der benötigten Bibliotheken³ zu geben. (Alle **SOAAP-System** im weiteren Inhalt dieses Artikels beziehen sich alle **SOAAP-System** auf die vom Prof. Robert Patzke).

4.3.1 Demonstrationsaufbau des SOAAP-Systems

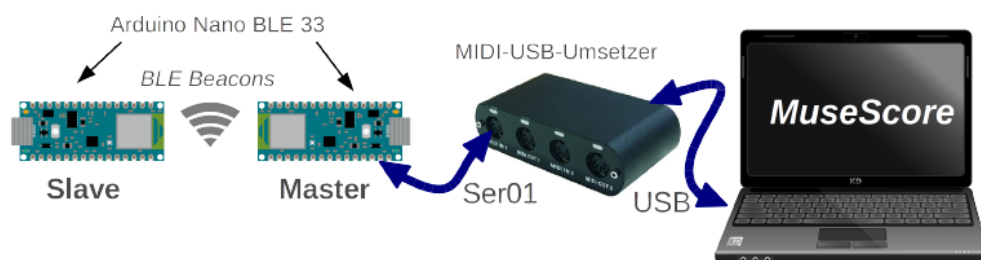


ABBILDUNG 4.1: Demonstrationsaufbau des SOAAP-Systems [5]

In der Abbildung 4.1 ist der Demonstrationsaufbau des SOAAP-Systems mit Funkübertragung dargestellt, die aus vier Teilen besteht, nämlich *Slave*, *Master*, *MIDI-USB-Umsetzer* und *Terminal*. Es werden die aktuellen Werte des Eulerwinkel (Roll-, Pitch- und Yawwinkel) auf einem Arduino Nano BLE 33 (Slave) über Funk (BLE-Beacons) an einen anderen Arduino Nano BLE 33 (Master) übertragen. Dort werden die Messwerte auf Noten abgebildet, getrennt nach Notenhöhe und Notenhöhe, und über eine serielle Schnittstelle

³die alle Bibliotheken sind vom Prof. Robert Patzke selbst unter dem Thema *Dezentrale Heimautomatisierung (DHA)* entwickelt und sind für allgemeine Verwendung (Lizenz Creative Commons) auf GitHub zur Verfügung gestellt. URL: <https://github.com/RobertPatzke/homeautomation>

(MIDI) und einen MIDI-USB-Umsetzer an ein Terminal (Notebook) mit leistungsfähiger Audioausgabe (DAW, Virtuelle Instrumente oder Synthesizer) geschickt [5]. Der MIDI-USB-Umsetzer ist in der Tat ein MIDI-Interface, das ein MIDI 5DIN Interface in ein USB-Interface um wandelt. Weiterführende Informationen über den Aufbau des MIDI-USB-Umsetzer im SOAAP findet man im Unterabschnitt 4.3.2.

Die folgende Abbildung zeigt den vereinfachten Datenschema des SOAAP-Systems.

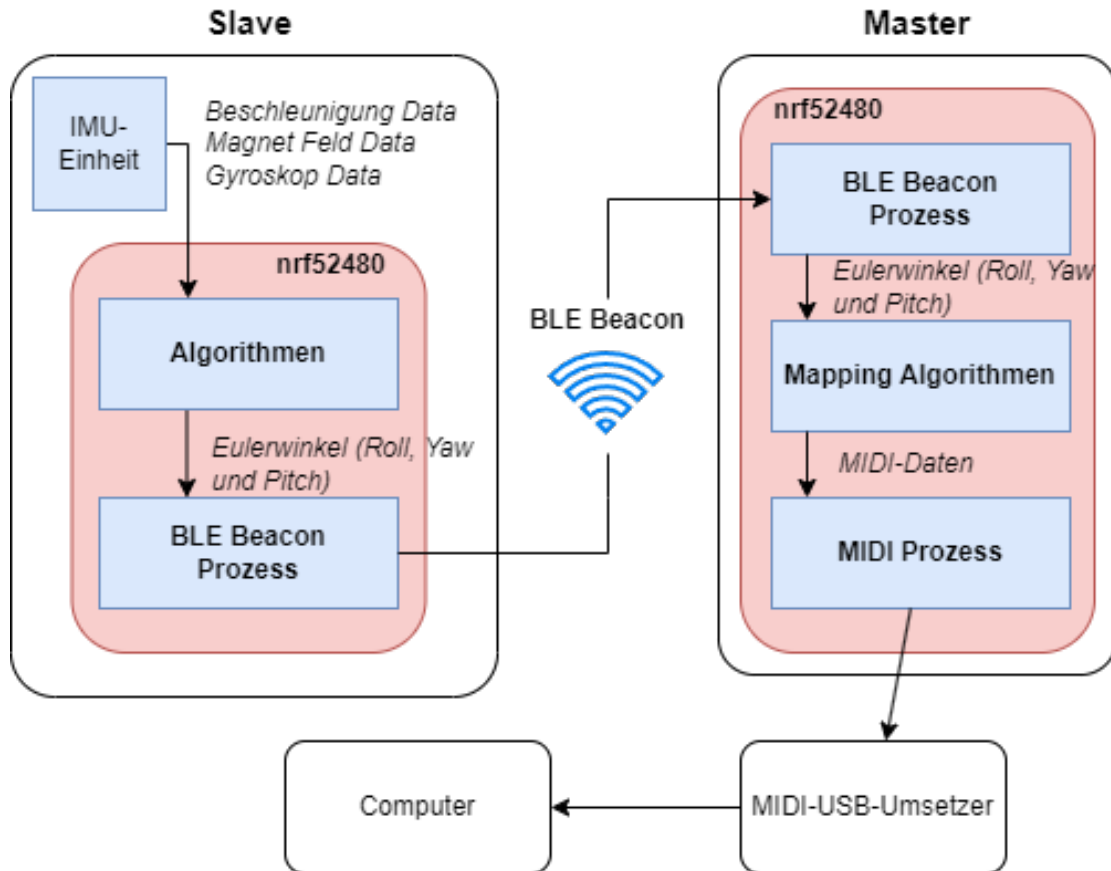


ABBILDUNG 4.2: Datenschema des SOAAP-Systems

Im Slave liest der *nrf52840*-Chips die Messdaten (Beschleunigung Data, Magnetfeld Data und Gyroskop Data) von der IMU-Einheit (eingebettete Sensor LSM9D5) über *I2C* Kommunikation-Schnittstelle und berechnet unter Anwendung eines speziellen Algorithmus⁴ die Eulerwinkel. Die Eulerwinkelsdaten werden vom *nrf52840*-Chip im Master über Funk (BLE Beacon) abgegriffen und mithilfe des Mapping-Algorithmus (über Mapping-Algorithmus siehe Unterabschnitt 4.5.2) in MIDI-Daten umgewandelt. Die MIDI-Daten

⁴Der Algorithmus wird hier nicht im Detail beschrieben, jedoch kann der genaue Code unter folgender URL eingesehen werden: <https://github.com/RobertPatzke/homeautomation/tree/developer/libraries>

werden im MIDI-Prozess als MIDI-Telegramm in einem Puffer gespeichert und über das *UART*-Protokoll am Computer übertragen.

Die Abbildung 4.2 zeigt einen vereinfachten Datenverlauf, eine detailliertere Codestruktur (Klasse Struktur) wird im Unterabschnitt 4.3.3 erläutert.

4.3.2 MIDI-USB-Umsetzer

Darstellung der Struktur:

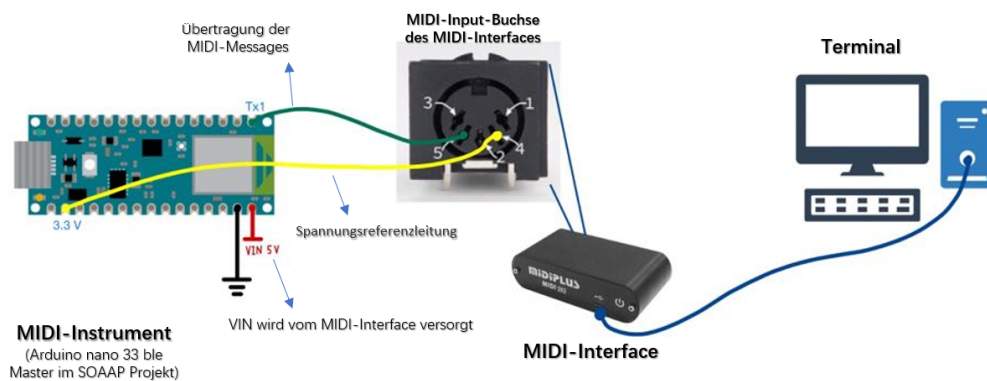


ABBILDUNG 4.3: Umbaustruktur des Arduinos und MIDI-Interfaces

In dem Projekt kann der Arduino nano 33 ble als ein MIDI-Instrument betrachtet werden, der generiert MIDI-Daten und hat einen MIDI-Output. Da die Programmieren-USB-Schnittstelle ist besetzt, um den Arduino Board zu programmieren oder Debugnachrichten über UART an PC zu schicken, werden MIDI Daten über Tx1 Pin (MIDI-Output) ausgesendet. Wie bei anderen MIDI-Instrumenten wird der MIDI-Output von Arduino nano 33 ble mit dem MIDI-Input des MIDI-Interfaces verbunden. Der Tx1 Pin wird mit Pin 4 der DIN-Buchse über einen Leitungsdraht zusammen gelötet (siehe Bild 4.3 grüne Linie), Pin 3.3V mit Pin 5 der DIN-Buchse als Spannungsreferenzleitung (siehe Bild 4.3 gelbe Linie), und die beiden GND sollen auch zusammen verbunden werden. Darüber hinaus braucht Arduino eine 5V Energieversorgung vom Main-Board des MIDI Interfaces, der auch vom PC oder Raspberry Pi versorgt wird.

In dem Fall von diesem Projekt steht kein Schutzwiderstand bei dem PIN Tx1 vom Arduino nano Board, deswegen muss man eine 220 Ohm Widerstand zwischen PIN Tx1

von Arduino nano und PIN 5 der MIDI-IN-Buchse (siehe Abbildung 4.4 und 4.5). Andererseits wird der 220-Ohm-Widerstand am Ausgang eliminiert und durch eine Spannung von 3,3 V ersetzt (vom PIN 3.3V des Arduinos), um sicherzustellen, dass der Strom der gleiche ist wie in der ursprünglichen Schaltung.

in der ursprünglichen Schaltung:

$$I = \frac{U_{\text{Spannungsreferenzleitung}}}{3 * R_{\text{Schutzwiderstand}}} = \frac{5V}{3 * 220\Omega} = 7,5mA \quad (4.1)$$

in der Schaltung des Projekts:

$$I = \frac{U_{\text{Spannungsreferenzleitung}}}{3 * R_{\text{Schutzwiderstand}}} = \frac{3,3V}{2 * 220\Omega} = 7,5mA \quad (4.2)$$

Physisches Bild:

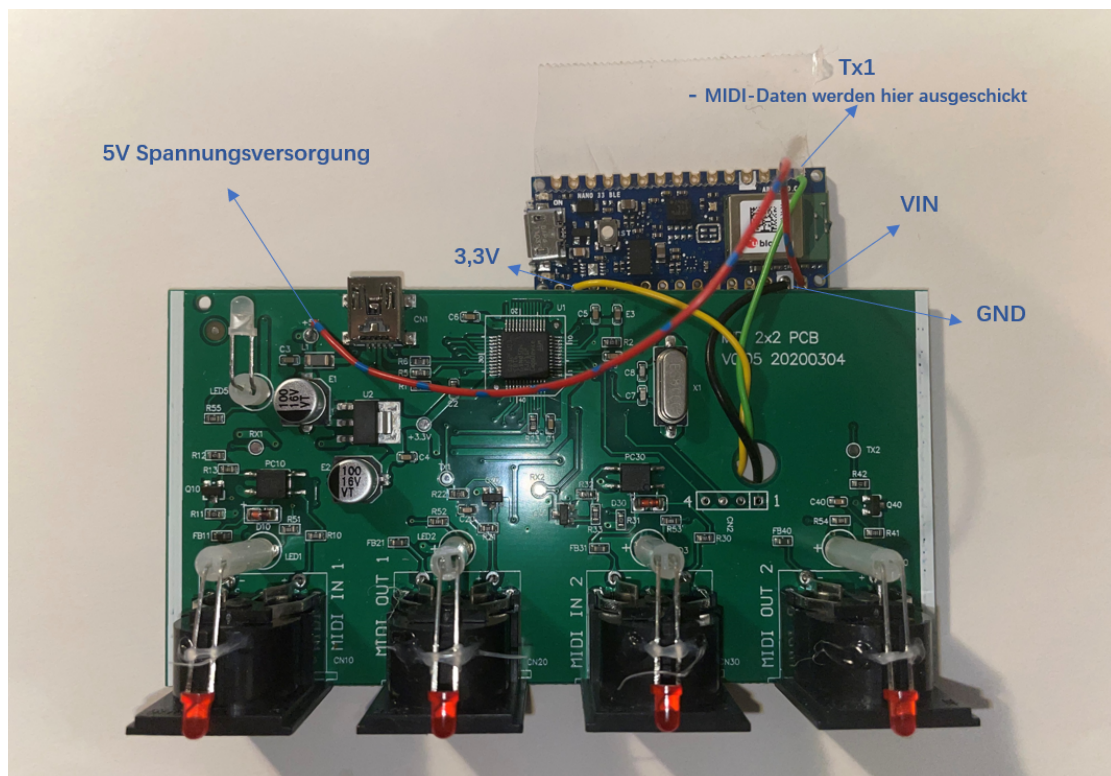


ABBILDUNG 4.4: Vorderseite des MIDI-USB-Umsetzer

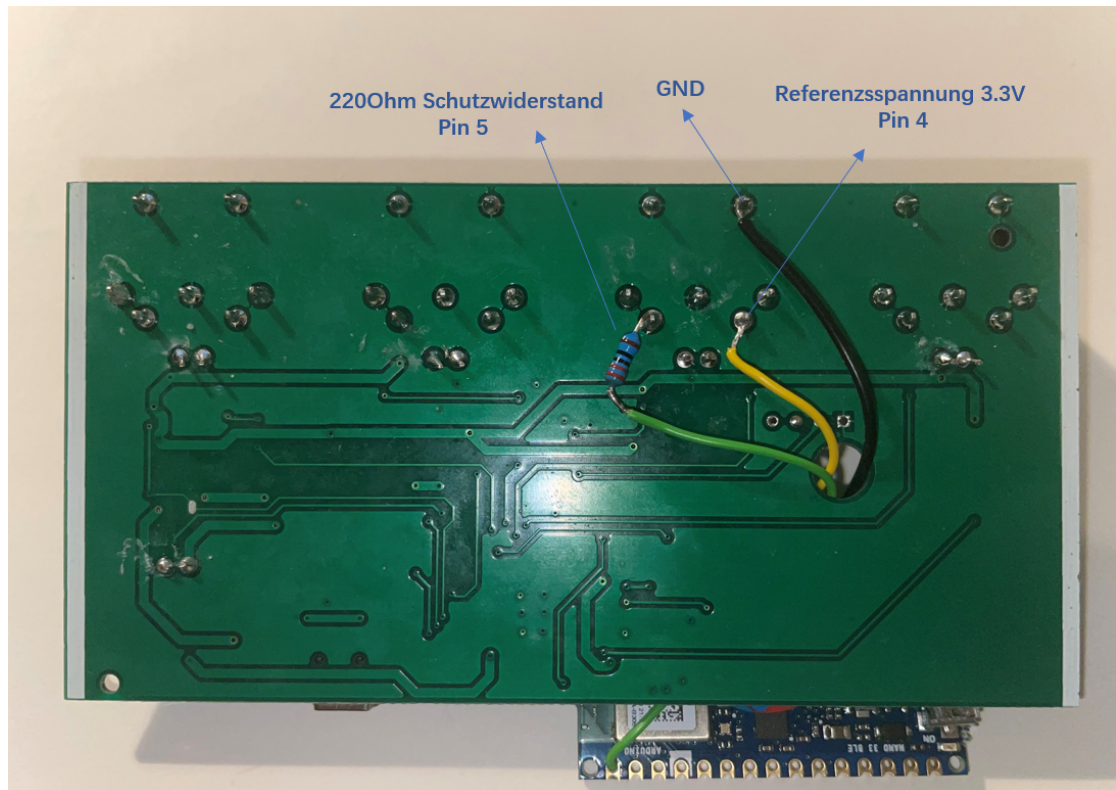


ABBILDUNG 4.5: Rückseite des MIDI-USB-Umsetzer

4.3.3 Klassen im Master

Die im Master verwendeten Klassen werden in diesem Abschnitt erläutert.

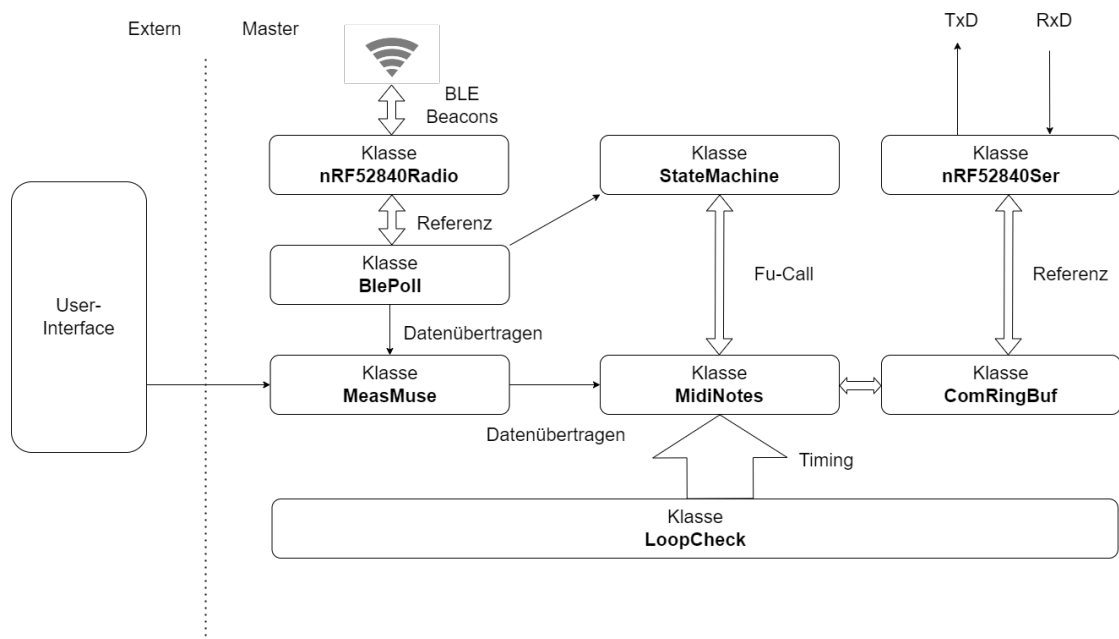


ABBILDUNG 4.6: Im Master verwendete Klassen

Im Abbildung 4.6 sind wichtige Klassen zur Datenübertragung zwischen Master und Slave (Polling) und zur Verarbeitung der Messwerte (MIDI-Steuerung) aufgeführt. Mit der Klasse *LoopCheck* wird ein Zeitrahmen für Software-Timer in die Arduino-Hauptschleife *loop()* eingebaut. Damit wird ein einfacher Ersatz für ein Echtzeitbetriebssystem geschaffen. Die Klasse *StateMachine* bietet verschiedene Methoden, um Zustandsmaschine zu konstruieren und zu steuern, wie z.B. Zustandsübergänge, Zeitabläufe, etc). Über die von dieser Klasse zur Verfügung gestellten Timer werden die *run()*-Funktionen der Zustandsmaschinen aufgerufen. Alle Prozesse der anderen Klasse werden unter Anwendung der Klassen *LoopCheck* und *StateMachine* durchgeführt.

Die Steuerung der Funkübertragung auf der Ebene des inhaltsunabhängigen Datenaustausches (Schichten 1 und 2 des entsprechenden ISO-Referenzmodells) erfolgt im Interruptbetrieb in der Klasse *nRF52480Radio*. Die Klasse *BlePoll* greift darauf zu und steuert den Datenverkehr zwischen Master und Slave. Diese Klasse verfügt über eine eigene Zustandsmaschine. Diese ermöglicht es dem Master, Abfragen (Polling) zu senden und Antworten von den Slaves zu empfangen. Im Slave erfolgt die Reaktion ebenfalls in der Klasse *BlePoll*, allerdings werden andere Zustände durchlaufen als beim Master.

Darüber hinaus ist im Master eine Anwender-Zustandsmaschine enthalten. Damit werden die von den Slaves eingeholten Messwerte auf MIDI-Kommandos abgebildet. Zur Unterstützung dient die beiden Klasse *MeasMuse* und *MidiNotes*. In der Klasse *MeasMuse* gibt es eine Reihe von Parametern, die vom Benutzer eingestellt werden können, und ein paar Mapping-Funktionen. Die Aufgabe der Mapping-Funktion besteht darin, die von den Slaves eingeholten Messwerte zu übernehmen und den MIDI-Value zurückzugeben (Eine detaillierte Beschreibung über die Klasse *MeasMuse* folgt im Unterabschnitt 4.5). Die Klasse *MidiNotes* abwickelt mit einer eigenen Zustandsmaschine den zeitlichen Ablauf eines über eine Note dargestellten Tones. Grundlage dafür ist die in der Musik übliche Variation eines Tones über das Anschwelle (*attac*), das Abklingen (*decay*), das Durchhalten (*sustain*) und das Ausklingen (*release*) sowie eine kurze Pause zum Übergang auf die nächste Note. Die Klasse *MidiNotes* behandelt zur Zeit Gleichklänge von maximal vier Noten (*Akkorde*). Eine einzelne Noten ist damit der Sonderfall eines Akkords mit nur einem Ton [5].

Die Klasse *MidiNotes* generiert Midi-Sequenzen und übergibt sie an die Klasse *ComRingBuf*. *ComRingBuf* bedient unterschiedliche serielle Schnittstellen, die über eine Referenz bei der Initialisierung angebunden werden. Hier wird die Klasse *nRF52480Ser* eingesetzt.

4.4 Eulerwinkel und MIDI-Value im SOAAP

Wie im Unterabschnitt 4.3.1 erwähnt, der Prozess im Slave die Aufgabe übernimmt, aus den Messdaten des eingebetteten IMU-Sensors die Roll-, Pitch- und Yawwinkel zu berechnen. Nach der Master die drei Winkelwerte empfangen hat, wendet er einen bestimmten Algorithmus an, um ein MIDI-Value (Eine Erläuterung des Midi-Values erfolgt im weiteren Verlauf dieses Unterabschnitt.) abzuleiten. In diesem Unterabschnitt wird erklärt, wie folgende Benutzer die Töne über das SOAAP-System steuern können. Der Slave wird während der Benutzung auf dem Handdrücken des Benutzers in einer festen Ausrichtung befestigt. Wenn die Hand gedreht wird, ändern sich die Werte der drei Winkel entsprechend. Vorausgesetzt, der Slave wird gemäß folgender Anweisung auf der Hand platziert (siehe Abbildung 4.7).

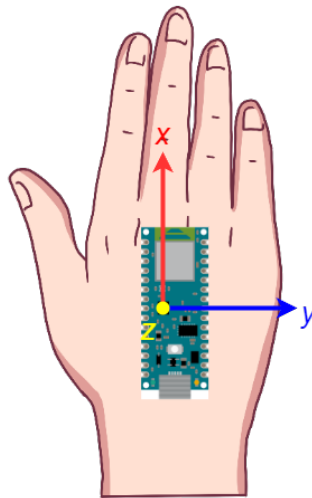


ABBILDUNG 4.7: Platzierung von dem Slave und der Hand

Die drei Drehachsen (x , y und z), welche im Slave eingestellt sind, werden in der Abbildung 4.7 dargestellt. Die positive x -Achse ist durch die Richtung des Fingers definiert. Die positive y -Achse verläuft in derselben Ebene wie die Handfläche und ist senkrecht

zur x-Achse nach rechts ausgerichtet. Die positive z-Achse verläuft senkrecht zur Handfläche und nach oben. Im folgenden Diagramm 4.8 wird die Zusammenhang zwischen der Richtung der Handaktivität und dem Eulerwinkel veranschaulicht.

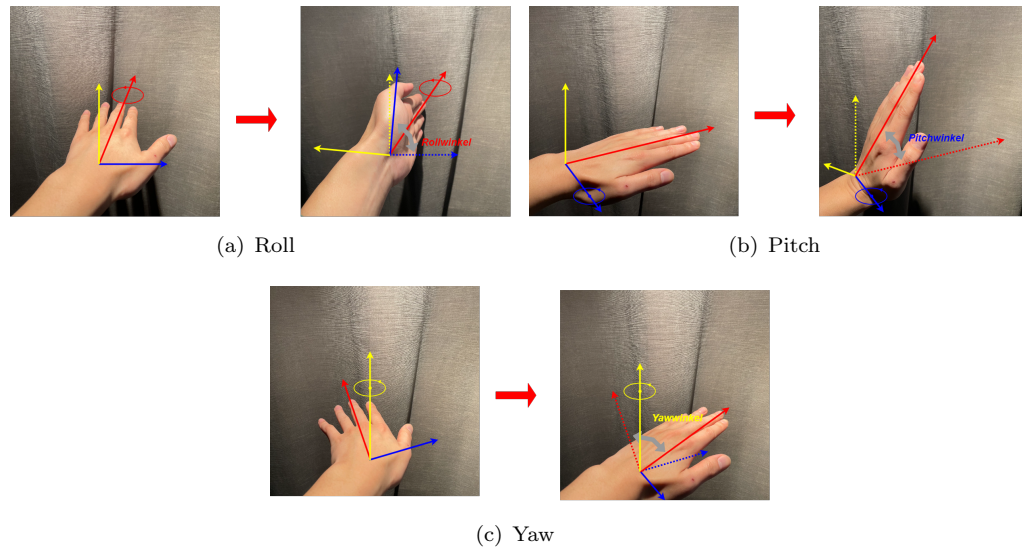


ABBILDUNG 4.8: Zusammenhang zwischen der Richtung der Handaktivität und dem Eulerwinkel

Obwohl Euler-Winkel einen Bereich von 0 bis 360 Grad abdecken können, ist es für das menschliche Handgelenk unmöglich, diesen gesamten Bereich (von 0 bis 360 Grad) in jeder Richtung zu erreichen. Als Ausgangsposition wird angenommen, dass die Handfläche parallel zur Horizontalen liegt. Und wenn die Hand wie in Abbildung 4.8 gezeigt bewegt wird, ergibt sich ein ungefährender Bewegungsbereich, der aus einer am Rollwinkel Bewegung von -45 bis 180 Grad, einer am Pitchwinkel Bewegung von -90 bis 90 Grad und einer am Yawwinkel Bewegung von -45 bis 45 Grad besteht. Es sei darauf hingewiesen, dass dieser Bereich von Person zu Person unterschiedlich sein kann. Aus diesem Grund ist es notwendig, den gültigen Bereich sowohl für die Eulerwinkel (für die Handbewegung) zu definieren. Um den gültigen Bereich zu bestimmen, sind für Roll-, Pitch- und Yawwinkel sowohl ein Mindest- als auch ein Höchstwert erforderlich.

Im Folgenden werden die Töne betrachtet. Ein Musikstück kann in einzelne Noten zerlegt werden, die drei wichtigsten Merkmale von Noten sind Tonhöhe, Lautstärke, Notenlänge (Rhythmus). Das MIDI-Value hier bezieht sich auf den Wert im Datenbyte des MIDI-Telegramms⁵. Im *Note On* MIDI-Telegramm steuert das erste Datenbyte die Tonhöhe und das zweite Datenbyte die Lautstärke. Das Intervall zwischen *Note On* und *Note*

⁵Mehr Informationen über MIDI-Telegramm siehe im Webseite:
<https://www.amazona.de/workshop-das-midi-datenformat-einfach-erklart/>

Off bestimmt. Die Tonhöhen- und Lautstärkewerte liegen im Bereich von 0 bis 127 (Ein-Byte-Bereich). Für die Notenlänge werden 14 Längen definiert (siehe Abbildung 4.9).

nti0	nti1	nti2p	nti2	nti4p	nti4	nti8p	nti8	nti16p	nti16	nti32p	nti32	nti64p	nti64
Doppel-note	ganze Note	punktierte halbe Note	halbe Note	punktierte viertel Note	viertel Note	punktierte achte Note	achte Note	punktierte 1/16 Note	1/16 Note	punktierte 1/32 Note	1/32 Note	punktierte 1/64 Note	1/64 Note

ABBILDUNG 4.9: List der 14 Notenlängen

MIDI-Daten können ebenfalls auf gültige Bereiche eingestellt werden, wie beispielsweise Maximal- und Minimalwerte für die Tonhöhe, Lautstärke und Notenlänge.

Es ist erforderlich, festzustellen, welche Notenmerkmale (Tonhöhe, Lautstärke oder Notenlänge) von welchem Eulerwinkel (Roll-, Pitch- oder Yawwinkel) gesteuert werden sollen. Ein Beispiel dafür ist die Verwendung vom Rollwinkel zur Steuerung der Tonhöhe, Pitchwinkel zur Steuerung der Lautstärke und Yawwinkel zur Steuerung der Notenlänge, sodass der Ton höher wird, wenn die Hand nach oben gehalten wird, lauter wird, wenn die Hand nach rechts rollt, und sich die Notenlänge ändert, wenn die Hand nach links oder rechts schwingt.

Der Algorithmus zur Umrechnung von Eulerwinkel in MIDI-Value wird im nächsten Unterabschnitt 4.5.2 erläutert.

4.5 Klasse MeasMuse

4.5.1 Übersicht

Der Name *MeasMuse* besteht aus zwei Teilen, *Meas* und *Muse*. *Meas* ist eine Abkürzung von *Measure*, das sich auf den Messwerte vom Slave bezieht. *Muse* ist eine Abkürzung von *Musik* und bezieht sich auf die MIDI.

Mit der Klasse *MeasMuse* werden zwei Aufgaben erfüllt. Zunächst wird damit eine externe Schnittstelle ermöglicht, über die das UI auf das SOAAP-System zugreifen kann. Dies ist jedoch ein einseitige Zugreif. *MeasMuse* kann nur die aktualisierte Daten von dem UI empfangen (siehe Abbildung 4.6). Die zweite Aufgabe ist, die Algorithmus zu entwickeln, um die von den Slaves eingeholten Messwerte in MIDI-Value umzuwandeln.

Dabei ist zu beachten, dass die Messwerte vom Slave in der Tat die Daten (*Beschleunigung Data*, *Magnetfeld Data* und *Gyroskop Data*) vom Sensor *LSM9D5* sind. Jedoch empfängt der Master den berechneten Eulerwinkel, wie im Unterabschnitt 4.3.1 erwähnt. In diesem Unterabschnitt befassen sich ausschließlich mit den Daten, die vom Master empfangen werden, eigentlich Eulerwinkel.

4.5.2 Mapping Algorithmen

Derzeit sind zwei Algorithmen verfügbar:

Direkte Wertübertragung: Kein Algorithmus

Lineare Wertübertragung: Der Eulerwinkel (Roll, Pitch und Yaw) von Slaves werden als lineare Funktion auf das MIDI-Value abgebildet. Die Gleichung lautet wie folgt:

$$f_{MIDI_Value}(x) = kx_{Eulerwinkel} + offset \quad (4.1)$$

Die Variable x ist der Eulerwinkel (der Eulerwinkel von Slaves) und $f(x)$ ist das MIDI-Value von x , nachdem er linear berechnet wurde. Und dabei bezeichnet k den Wert für die Steigung und b den $f(x)$ -Achsenabschnitt. Das Graph der lineare Funktionsgleichung mit variablen Einschränkungen sieht man unten.

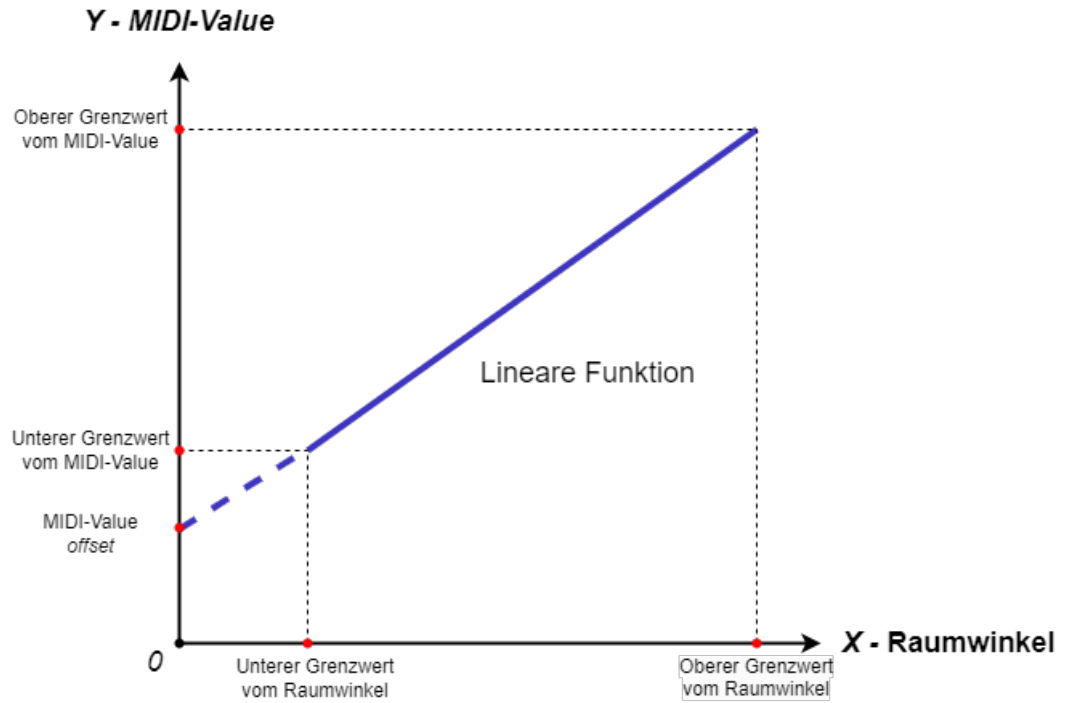


ABBILDUNG 4.10: Graph der lineare Funktionsgleichung

Die Grenzwerte der räumlichen Winkel und des MIDI-Values werden durch den Benutzer festgelegt. Die Steigung k lässt sich mit den Grenzwerte ausrechnen, siehe die Formel unten.

$$k_{Steigung} = \frac{\text{obererGrenzwertvomMIDI_Value} - \text{untererGrenzwertvomMIDI_Value}}{\text{obererGrenzwertvomEulerwinkel} - \text{untererGrenzwertvomEulerwinkel}} \quad (4.2)$$

offset des MIDI-Values kann ermittelt werden.

$$\begin{aligned} \text{offset}_{MIDI_Value} = & \text{UntererGrenzwertvomMIDI_Value} \\ & - k_{Steigung} * \text{UntererGrenzwertvomEulerwinkel} \end{aligned} \quad (4.3)$$

Auf die gleiche Art und Weise wird die Variable offset zum Eulerwinkel addiert, und der wird durch den Benutzer angegeben.

$$\text{eingegebenerEulerwinkelwert} = \text{gemessenerEulerwinkelwert} + \text{offset}_{\text{Eulerwinkel}} \quad (4.4)$$

Die Beziehung zwischen der Abbildung von Eulerwinkeln zum MIDI-Value ergibt sich aus der Zusammenfassung der drei Grundlagen 4.1, 4.2, 4.3 und 4.4.

$$\begin{aligned} \text{MIDI_Value} &= k_{\text{Steigung}} * \text{eingegebenerEulerwinkelwert} + \text{offset}_{\text{MIDI_Value}} \\ &= \frac{\text{obererGrenzwertvomMIDI_Value} - \text{untererGrenzwertvomMIDI_Value}}{\text{obererGrenzwertvomEulerwinkel} - \text{untererGrenzwertvomEulerwinkel}} \\ &\quad * (\text{gemessenerEulerwinkelwert} + \text{offset}_{\text{Eulerwinkel}}) \\ &\quad + \text{UntererGrenzwertvomMIDI_Value} \\ &\quad - \left(\frac{\text{obererGrenzwertvomMIDI_Value} - \text{untererGrenzwertvomMIDI_Value}}{\text{obererGrenzwertvomEulerwinkel} - \text{untererGrenzwertvomEulerwinkel}} \right. \\ &\quad \left. * \text{UntererGrenzwertvomEulerwinkel} \right) \end{aligned} \quad (4.5)$$

Lineare Funktion ist am einfachsten zu berechnen, um ein Mapping zu erstellen. Der Mapping-Algorithmus kann auch andere mathematische Funktionen verwenden, wie z.B. mit der Exponentialfunktion, der Logarithmusfunktion oder der Potenzfunktionen mit dem höheren Exponent usw.

4.5.3 Parameter in der Klasse *MeasMuse*

In diesem Unterabschnitt werden drei wichtige Enumerationen dargestellt. Sie sind *Meas2Midi*, *MeasMap* und *MidiOpMode*. Die Enumeration *Meas2Midi* ist

```
enum Meas2Midi
{
    Nothing,          // keine Zielzuweisung
    NoteType,         // Zuweisung Notentyp (Laenge)
    NoteVal,          // Zuweisung Notenwert (Hoehe)
    NoteVel,          // Zuweisung Anschagstaerke
```

```

NrM2M          // Anzahl fuer Array, return 4
};

```

In der Enumeration *Meas2Midi* sind drei Merkmale der Note, nämlich *NoteType* für Notenlänge (BMP), *NoteVal* für Notenhöhe und *NoteVel* für Anschlagstärke. Sie werden den drei Eulerwinkeln (Roll, Pitch und Yaw) zugewiesen, z.B. Roll für den Notenlänge, Pitch für die Notenhöhe und Yaw für die Anschlagstärke. Die Zielzuweisung dürfen nicht dupliziert werden, da es sonst bei der Umwandlung der MIDI-Telegramm zu Konflikten kommen kann. *Nothing* bedeutet keine Zielzuweisung, der entsprechende Eulerwinkel hat keine Kontrolle über die MIDI-Telegramme. *NrM2M* ist die Anzahl gleich 4, die dient zur Festlegung der Länge eines Arrays für *MidiArea*. Nächste Enumeration *MeasMap* sieht man unten

```

enum Meas2Midi
{
    OneToOne,      // Direkte Wertuebertragung
    BiLinear,      // Linear auf Linear
    NrMM           // Anzahl fuer Array, return 2
};

```

In der Enumeration *MeasMap* sind zwei Mapping-Algorithmen, *OneToOne* bedeutet die direkte Wertübertragung, *BiLinear* ist dann die lineare Wertübertragung. Dies sind die einzigen beiden Algorithmen, die derzeit zur Verfügung stehen, aber Mapping-Algorithmen sind nicht nur auf diese beiden beschränkt, und so können weitere was Neues erprobt und entwickelt werden. *NrMM* ist die Anzahl gleich 3. Die dritte Enumeration *MidiOpMode* ist

```

enum _MidiOpMode
{
    momIdle,       // stand-by
    momSequence,   // nicht-kontinuierlicher Ton
    momRunDelta    // kontinuierlicher Ton
};

```

In der Enumerationen *MidiOpMode* sind drei Spielmodi, *monIdle* bedeutet der Zustandsmaschine, der MIDI-Telegramme generiert, in einem Standby-Zustand bleibt, in dem Fall wird keine MIDI-Telegramme ausgegeben. Im *momSequence*-Modus ist der Ton wie bei einem Klavier intermittierend. Und im *momRunDelta*-Modus ist der Ton so kontinuierlich wie bei einer Geige. Da es bei einem kontinuierlichen Ton kein Rhythmuskonzept gibt, ist es in diesem Modus nicht notwendig, die Notenlänge einzustellen.

der MIDI-Value kann als obere, untere und Offset-Werte eingestellt werden. Die zugehörige Datenstruktur ist

```
typedef struct _MidiArea
{
    byte  low;      // unterer Grenzwert
    byte  high;     // oberer Grenzwert
    byte  offset;   // offsetswert
} MidiArea, *MidiAreaPtr;
```

Die Datenstruktur des nach dem Mapping erhaltenen MIDI-Value ist wie folgt

```
typedef struct _MidiResult
{
    Meas2Midi type;    // Zielzuweisung (Notenlaenge, Notenhoehe und
                      // Anschlagstaerke)
    byte      value;   // neues MIDI-Value
    bool      newVal;  // Ob das Midi-Vlaue aktualisiert wurde
} MidiResult, *MidiResultPtr;
```

4.5.4 eingebettete Klasse *Posture2Midi*

Die Klasse *Posture2Midi* ist eine eingebettete Klasse in *MeasMuse*. Mit dem werden die Mapping-Algorithmen implementiert. Die Instanz der Klasse *Posture2Midi* wird wie unten eingerichtet.

```
Posture2Midi  config[NrOfChannelsMM];
              // Konfiguration der Abbildung
```

NrOfChannelsMM ist die maximale Anzahl des Channels (Da die maximale Anzahl von Kanälen im Midi-Protokoll 16 beträgt, ist der *NrOfChannel* theoretisch maximal 16).

Die zugehörige Konfigurationsdaten sind

```

float      offsetRoll;
float      borderLowRoll;
float      borderHighRoll;
float      koeffRoll;
float      gapRoll;
bool       doGapRoll;
Meas2Midi  aimRoll;

float      offsetPitch;
float      borderLowPitch;
float      borderHighPitch;
float      koeffPitch;
float      gapPitch;
bool       doGapPitch;
Meas2Midi  aimPitch;

float      offsetYaw;
float      borderLowYaw;
float      borderHighYaw;
float      koeffYaw;
float      gapYaw;
bool       doGapYaw;
Meas2Midi  aimYaw;

int        midiOpMode;
MidiArea   midiArea[NrM2M];
bool       ready;
```

Die Konfigurationsdaten haben folgende Bedeutung:

offsetRoll, offsetPitch, offsetYaw	- offset vom Eulerwinkel Roll, Pitch und Yaw
borderLowRoll, borderLowPitch, borderLowYaw	- unterer Grenzwert vom Eulerwinkel Roll, Pitch und Yaw
borderHighRoll, borderHighPitch, borderHighYaw	- oberer Grenzwert vom Eulerwinkel Roll, Pitch und Yaw
koeffRoll, koeffPitch, koeffYaw	- Koeffizient (Steigung k) für das lineare Mapping vom Eulerwinkel Roll, Pitch und Yaw
gapRoll, gapPitch, gapYaw	- Schwellenwert vom Eulerwinkel Roll, Pitch und Yaw
doGapRoll, doGapPitch, doGapYaw	- ein Boolean für die Aktivierung des Schwellenwert
aimRoll, aimPitch, aimYaw	- Zielzuweisung (Notenlänge, Notenhöhe oder Anschlagstärke am Eulerwinkel Roll Pitch und Yaw
midiOpMode	- Spielmodi
ready	- 1 - Spielmodus ist eingestellt, 2 - nicht
midiArea[NrM2M]	- Array für die Grenzwerte vom MIDI-Value in drei Typen (Notenlänge, Notenhöhe und Anschlagstärke)

Die List der Methoden ist

```
void setKoeffRoll(MeasMap map);  
void setKoeffPitch(MeasMap map);  
void setKoeffYaw(MeasMap map);
```

```
int  getResultRoll(MidiResultPtr refResult, float measValue);
int  getResultPitch(MidiResultPtr refResult, float measValue);
int  getResultYaw(MidiResultPtr refResult, float measValue);
```

Die Ermittlung des Koeffizient für das lineare Mapping sind mit der ersten drei Methoden implementiert. Der Koeffizient ist die Steigung, die im Unterabschnitt [4.5.2](#) erläutert wird. Der Parameter *map* ist ein Datentype *MeasMap* und hat die Bedeutung, welche Mapping-Algorithmus zu wählen. z.B. wie

```
setKoeffRoll(BiLinear);
setKoeffPitch(BiLinear);
setKoeffYaw(BiLinear);
```

Der lineare Algorithmus wird durch die letzten drei Methoden in der List der Methoden implementiert. Die Parameter sind

refResult	- Ausgabeparameter, eine Zeiger von dem Struct <i>_MidiResult</i> , zeigt auf den Parameter, in der das neue MIDI-Value gespeichert
measValue	- Eingabeparameter, aktuelle Eulerwinkelwert

4.5.5 Konfigurationsfunktion und Anwenderfunktion

Die List der Konfigurationsfunktion ist

```
void setRollArea(int channel, float offset, float min, float max);
void setRollGap(int channel, float gap);
void setPitchArea(int channel, float offset, float min, float max);
void setPitchGap(int channel, float gap);
void setYawArea(int channel, float offset, float min, float max);
void setYawGap(int channel, float gap);
void setAims(int channel, Meas2Midi aimRoll, Meas2Midi aimPitch,
```

```
Meas2Midi aimYaw);

void setMidiArea(int channel, Meas2Midi midi, byte low, byte high);
void setMapping(int channel, MeasMap mapRoll, MeasMap mapPitch,
MeasMap mapYaw);

void setOpMode(int channel, int opMode);
```

Die Parameter in *Posture2Midi* können über diese Konfigurationsfunktionen zugegriffen werden, alle Daten wird nach dem Channel unterschiedet.

```
int resultRoll(int channel, MidiResultPtr refResult,
float measValue);
int resultPitch(int channel, MidiResultPtr refResult,
float measValue);
int resultYaw(int channel, MidiResultPtr refResult,
float measValue);
```

In den drei Funktionen rufen den Methode (siehe unten) auf, die dazu dient, das neue MIDI-Value auf den verschiedenen Kanälen zu erhalten (am Beispiel des Eulerwinkels Roll)

```
config[channel].getResultPitch(refResult, measValue);
```

Die letzte Anwenderfunktion gibt den Abspielmodus zurück, wenn der Abspielmodus nicht eingestellt ist, wird -1 zurückgegeben.

```
int getOpMode(int channel);
```

Kapitel 5

Entwicklung des Userinterfaces

5.1 Systemkonzept des Userinterfaces

5.1.1 Motivation des Userinterfaces

Wie bereits in der Einleitung erwähnt, ist diese Arbeit Teil des SOAAP-Projekts und basiert auf dem SOAAP-System, das von Prof. Robert Patzke entwickelt wurde. Es gibt eine Reihe von verstellbaren Parameter im System, die jedoch derzeit nur im Programm durch Aufruf der entsprechenden Methoden geändert werden können, was für den Benutzer unvernünftig ist. Die Idee dieser Arbeit ist es daher, eine grafische Benutzeroberfläche (GUI) für das System zu entwickeln. Gleichzeitig sollte das Framework für das Backend der Benutzeroberfläche implementiert werden, um die Kommunikation zwischen der Benutzeroberfläche und dem Master zu realisieren.

5.1.2 Problemstellungen und Anforderungen des Userinterfaces

Bei der Entwicklung dieses UIs, das die genannten Aufgaben im Unterabschnitt [5.1.1](#) erfüllt, treten viele Probleme auf und es ist notwendig, unterschiedliche Aspekte zu betrachten. Die Arbeit kann in zwei Hauptteile unterteilt werden, zum einen ist das grafische Design der Benutzeroberfläche und zum anderen ist die Entwicklung des Backend-Kommunikationsframeworks. Die Probleme werden auch aus diesen beiden Aspekten gestellt.

Die List der Problemstellung des Backend-Kommunikationsframeworks :

- Welche Kommunikationstechnik sollte zwischen dem SOAAP-System und dem UI gewählt werden?
- Wie sollte die Hardware-Module zu ergänzen bzw. zu modifizieren?
- Welche Entwicklungsplattform und Entwicklungsumgebung sollte für die Entwicklung des UI und GUI verwendet werden?
- Wie sollte Telegramm für die Kommunikation gestaltet sein?

Die List der Problemstellung des Benutzeroberfläche-Designs :

- Welche Features sollte die Benutzeroberfläche zur Verfügung gestellt werden?
- Welche UI-Komponenten sollten für die verschiedenen Parameter verwendet werden?
- Wie kann die Benutzeroberfläche verschönert werden und leichter nutzbar werden?

Bei der Bewältigung der oben genannten Probleme gibt es eine Priorität. Der Aufbau des Backend-Kommunikationsframeworks hat Vorrang vor der Entwicklung des Benutzeroberfläche-Designs. Was bei dieser Arbeit erreicht werden musste, ist die Implementierung des Kommunikationsframeworks, d. h. dass das UI Nachrichten an das SOAAP-System senden und von diesem empfangen kann. Darüber hinaus müssen die Prinzipien des Nachrichtenaustauschs untersucht werden, wann wird eine Nachricht vom UI und eine Nachricht von SOAAP gesendet wird. Wie wird eine Bestätigungsrückmeldung an den Absender sendet, nachdem die Nachricht vom Empfänger empfangen wurde.

Die Gestaltung der grafischen Benutzeroberfläche ist für diese Arbeit zweitrangig. Diese Aufgabe umfasst noch zwei weitere Teile. Die erste besteht darin, die grundlegenden Funktionen der Benutzeroberfläche zu implementieren, wie Buttons, Sliders und andere UI-Komponenten. Die zweite ist die Verschönerung der Benutzeroberfläche. Der erste Teil hat eine höhere Priorität als der zweite Teil.

Ein weiterer sehr relevanter Punkt ist die Lesbarkeit des Code in dieser Arbeit, es sollte möglichste gut sauber sein. Da es sich um ein Gemeinschaftsprojekt handelt, ist die Sauberkeit des Codes besonders wichtig. Wenn andere Studierende das System verwenden

oder auf dem Code aufbauen müssen, werden sie in der Lage sein, die Programmierstruktur und den Aufruf der Methoden im Code schnell verstehen. Daher ist eine gute Organisation des Codes mit Objekten und die Berücksichtigung der Nachhaltigkeit bei der Programmierung erforderlich.

5.1.3 System Schematik

In diesem Unterabschnitt wird ein vollständiges Schema beschrieben, das das SOAAP-System und UI umfasst. Das SOAAP-System Schematik ist im Unterabschnitt 4.3.1 dargestellt, das aus 4 Komponenten besteht, nämlich Slave(Nano 33 BLE), Master(Nano 33 BLE), MIDI-Umsetzer und Terminal(Notebook). Zusätzlich zu dieser Schematik wird ein neues WiFi-Modul(Esp8266) hinzugefügt, die für die reale Situation gewählte Hardware ist *Wemos D1 mini pro* (siehe Abbildung 5.1). Die Entscheidung für WiFi wird aufgrund der höheren Allgemeinheit dieser Technologie getroffen. Das Projekt ist so konzipiert, UI nicht nur auf einem Computer zu bauen, sondern die auch auf mobilen Geräten wie Smartphone oder Tablets verfügbar zu machen. Dies ist mit der WiFi-Kommunikation leichter zu erreichen, da sie die Einschränkungen der Verkabelung vermeidet und die Kommunikation von einem Punkt zu mehreren Punkte einfacher macht als mit drahtgebundenen Kommunikationstechnologien. Im Vergleich zu anderen drahtlosen Nahfunktechnologien, wie z.B. Bluetooth, bietet WiFi die Möglichkeit, SOAAP-System mit dem Internet zuzugreifen und hat aus Sicht der Funktionsentwicklung ein höheres Potenzial.

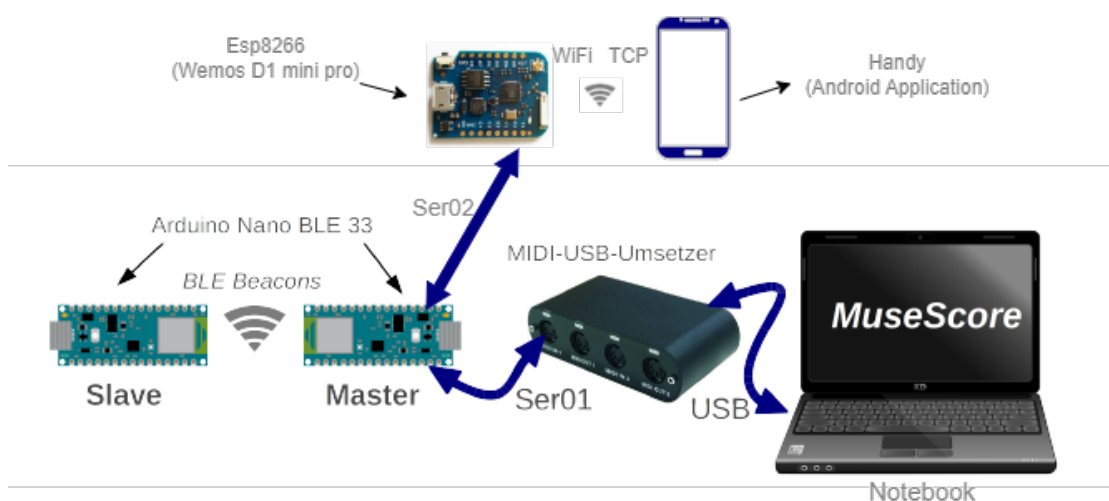


ABBILDUNG 5.1: Schematik des Userinterfaces und des SOAAP-Systems

Bei dieser Arbeit wird beschlossen, dass die UI-Applikation auf dem Smartphone entwickelt wird, das Betriebssystem ist Android. Der Esp8266 richtet einen Access Point ein, und wenn der Smartphone mit diesem verbunden ist, können die beiden Komponenten über das TCP-Protokoll kommunizieren. Der Esp8266 wird über UART an den Master angeschlossen, ein UART Schnittstellenport und Treiber werden am Master entwickelt. Die Rolle des Esp8266s in dieser Struktur besteht darin, Nachrichten vom Android-Applikation an den Master oder vom Master an das Android-Applikation weiterzuleiten. Die Nachrichten werden nicht im Esp8266 dekodiert, sondern im Master oder Android-Applikation.

5.1.4 Prozesse Erklärung

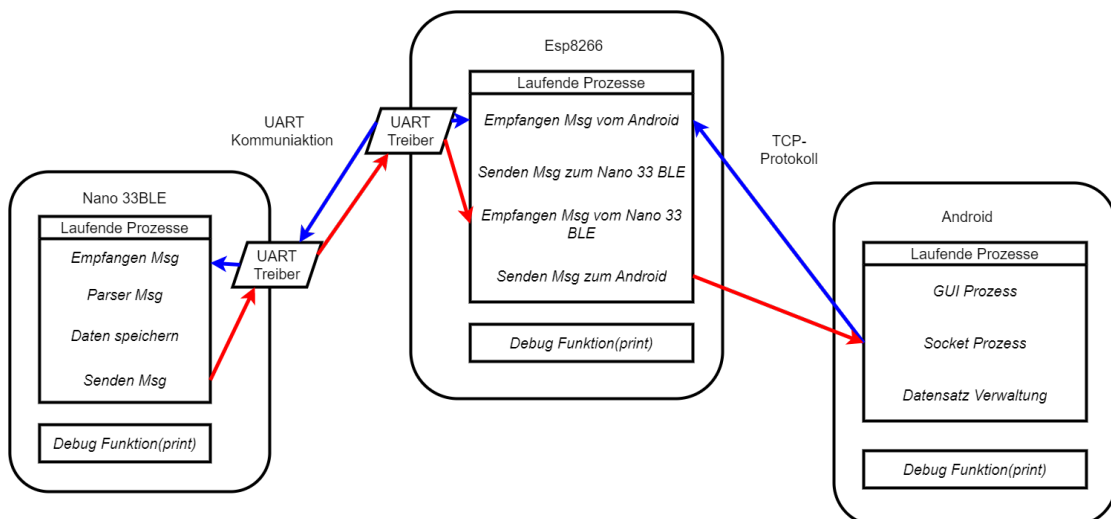


ABBILDUNG 5.2: Laufende Prozesse in Arduino Nano 33 BLE, Esp8266 und Android

Das Diagramm 5.2 zeigt allgemein die Prozesse, die in jedem Komponenten ablaufen. Im *Android* kann man das Programm in drei Prozesse unterteilt werden. Der Prozess *GUI* ist dafür verantwortlich, alle Komponenten auf der Benutzeroberfläche und die Callback-Funktion für alle UI-Komponenten zu aktualisieren. Nachdem die Benutzeroberfläche aktualisiert wurde, werden die neuen Daten im Prozess *Datensatz Verwaltung* gespeichert. Schließlich wird die Nachricht im Prozess *Socket Prozess* gesendet oder empfangen. Es gibt nur vier einfache Prozesse, die in der *Esp8266* enthalten sind, nämlich Empfangen und senden von der Seite *Android* oder *Nano 33 BLE*. Die Prozesse in *Nano 33 BLE* sind relativ komplex und es gibt viele Prozesse aus dem SOAAP-System, die in

nano ablaufen. Die für diese Arbeit relevanten Prozesse lassen sich in vier zusammenfassen. Im ersten Schritt werden die Nachrichten empfangen, weiter werden sie dekodiert und schließlich werden die dekodierten Daten in der entsprechenden Klasse (MeasMuse) des SOAAP-Systems gespeichert. Der Senden Prozess kann unabhängig von den drei oben genannten Prozessen ablaufen und ist nicht mit den drei sequentiell. Wenn der *Esp8266* mit dem *Nano 33 BLE* über UART kommunizieren soll, benötigt der nicht nur Hardwareunterstützung (z.B. der Chip hat ein UART-Kommunikationsmodul), sondern auch Softwareunterstützung. Der UART-Treiber vom *Nano 33 BLE* ist vom Prof. Robert Patzke selber entwickelt, die basiert auf den Chip *nRF52480* aus *Nordic*. Der UART-Treiber vom *Esp8266* ist von der Klasse *Serial*, die Arduino bietet.

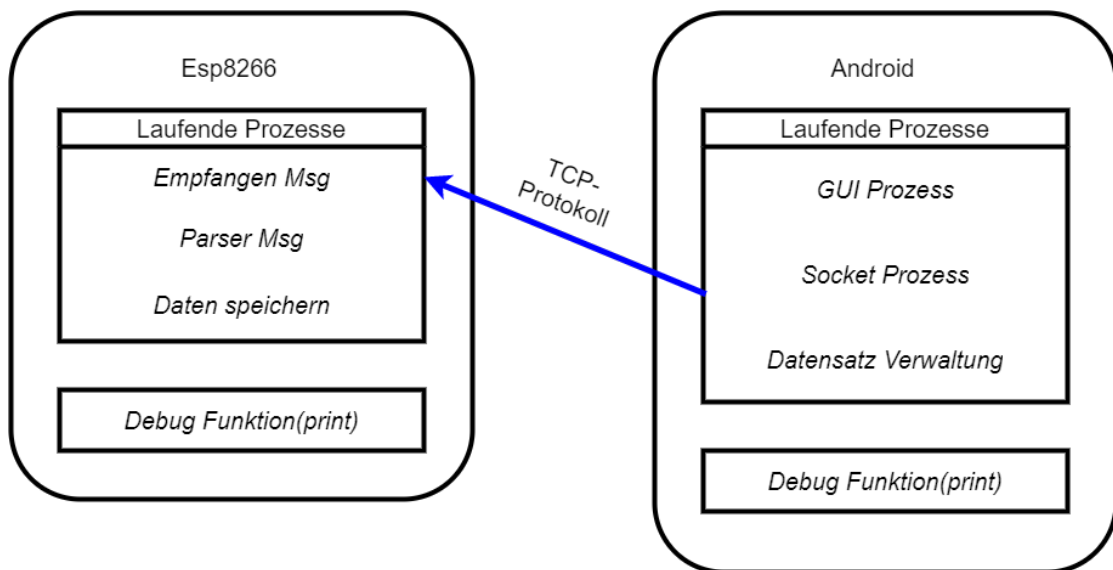


ABBILDUNG 5.3: Laufende Prozesse in Esp8266 und Android

Das Diagramm 5.3 zeigt alle Prozesse, die in dieser Arbeit implementiert sind. Da der UART-Treiber im *Nano 33 BLE* nicht erledigt wird, kann der *Esp8266* den nicht zugreifen. Daher werden in dieser Arbeit die oben (siehe Diagramm 5.2) gezeigten Prozesse, die in *Nano 33 BLE* ablaufen, nun in *Esp8266* ausgeführt.

5.1.5 Protokolle für Userinterface

Dieser Unterabschnitt erklärt die Telegramme¹, wie die in dieser Arbeit definiert ist. Da das *Esp8266* nur das WiFi-Modul als eine Station für die Weiterübertragung der

¹Das vorliegende Telegramm sind aus dem Code entstanden, den der Professor für das IoT-Experiment entwickelt hat, siehe im URL: https://moodle.hs-hannover.de/pluginfile.php/749884/mod_resource/content/4/EntwicklungMC.pdf

Nachrichten zur Verfügung stellt, besteht die Projekt-Schematik im Wesentlichen noch aus der Kommunikation zwischen dem Android-Applikation (Handy) und dem Master (Nano). Wenn man sich nur auf das Telegramm zwischen den beiden Geräten fokussieren, kann der Schematik auch wie folgt vereinfacht werden.

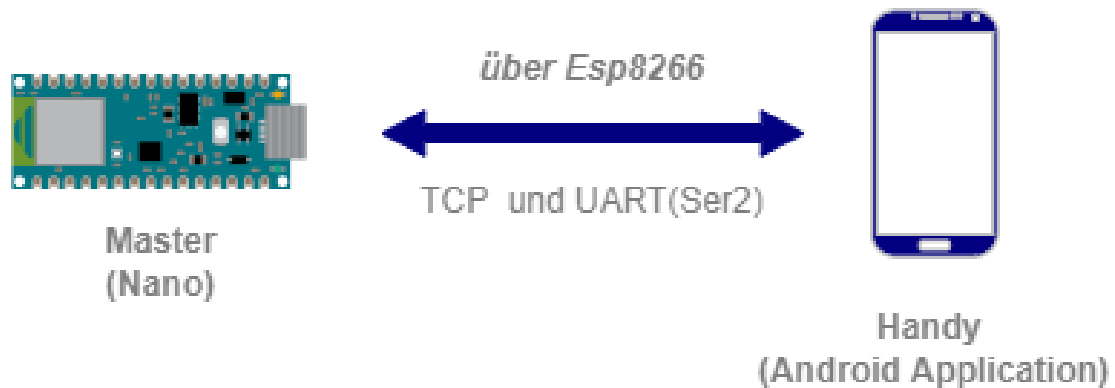


ABBILDUNG 5.4: Vereinfachter Schematik des Userinterfaces und des SOAAP-Systems

Die theoretische maximale Größe eines TCP-Pakets beträgt 65535 Bytes einschließlich der Header. In der Praxis werden Pakete in der Größe der kleinsten *MTU* (*Maximum Transmission Unit*) auf dem Pfad, d.h. in der Regel 1500 Bytes (für Ethernet) Daten einschließlich TCP (20 Byte) und IP-Header (20Byte). Wenn der Frame-Header ebenfalls hinzugefügt wird, so dass er bis zu 1518 Bytes beträgt. Wenn die Daten diesen Grenzwert überschreiten, kann dies einen Verlust von TCP-Datenpaketen verursachen. Für diese Arbeit ist diese Kapazität jedoch schon ausreichende. Der Aufbau des Telegramms vom Android-Applikation zum Master sieht man in der Abbildung 5.5.

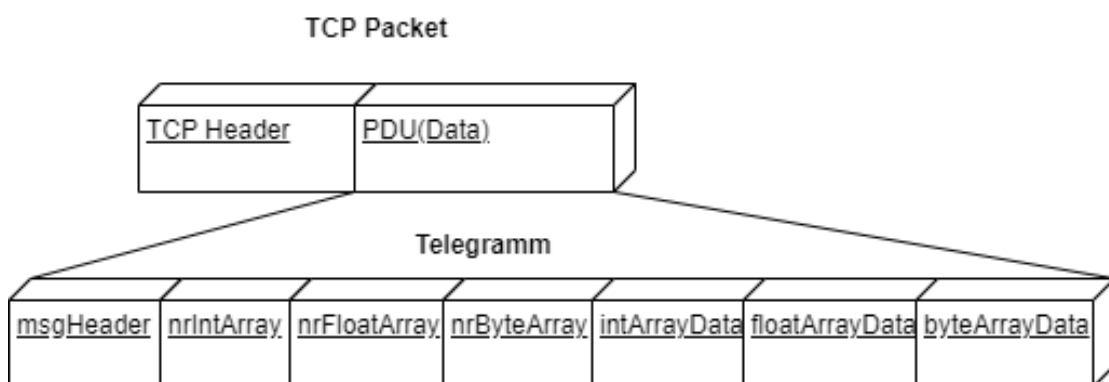


ABBILDUNG 5.5: Aufbau des Telegramms mit TCP-Protokoll

Das Telegramm besteht aus zwei Teile, *TCP Header* und *Data*. Um den *TCP Header* wird von der *Socket()* Methode aus dem Package *java.net.Socket* gekümmert. Die IP-Adresse und Port werden darin eingerichtet. In *PDU* ist die nützliche Nachrichten,

die weiterübertragen werden müssen. Als nächste befasst sich mit dem Teil *PDU* vom Telegramm.

In *msgHeader* wird der Name und die Version des Telegramms dargestellt, auch andere Informationen können hinzugefügt werden. *nrIntArray*, *nrFloatArray* und *nrByteArray* zeigen die Anzahl der aktuell zu sendenden Felder der Ganzzahlen, der Gleitkommazahlen und Bytes. *intArrayData*, *floatArrayData* und *byteArrayData* sind drei Felder, die die Werte von Ganzzahlen, Gleitkommazahlen und Bytes enthalten. Die Länge dieser Felder wird beim Bedarf vom Entwickler selber festgelegt. Der gesamte Teil *PDU* wird in das Stringformat konvertiert. Alle Werte werden mit dem Strichpunkt getrennt und das *PDU* endet immer mit zwei Doppelpunkten. Angenommen, dass das *PDU* mit den drei Felder, die alle mit Länge 2 sind, eingerichtet. d. h. *nrIntArray*, *nrFloatArray* und *nrByteArray* werden als 2 konfiguriert. Das *PDU* sieht so aus:

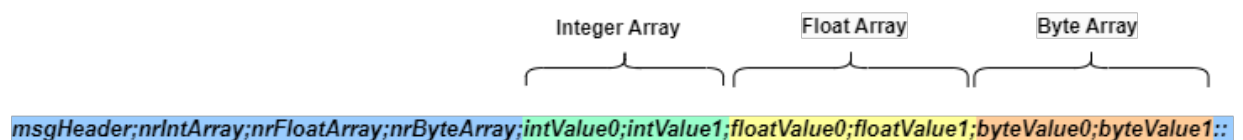


ABBILDUNG 5.6: Aufbau des PDU

Der nächste Schritt besteht darin, die TCP-Pakete mit *Wireshark* zu erfassen. Mit *Wireshark* lässt sich Datenverkehr auf unterschiedlichen Kommunikationsschnittstellen mitlesen, aufzeichnen und analysieren. Um die Aufzeichnung des Telegramms zu vereinfachen, wird die Länge jedes Felder auf 2 gesetzt (*nrIntArray*, *nrFloatArray* und *nrByteArray* gleich 2). Der *PDU* sieht so aus

```
msgHeader;2;2;2;intValue0;intValue1;floatValue0;floatValue1;
byteValue0;byteValue1::0
```

Esp8266 ist im Access Point gesetzt, und die IP-Adresse ist *192.168.4.1*, der Port ist *4210*. Die Verbindung des Telefons mit dem Access Point ist erforderlich. Nach dem Start der Android-Applikation sieht man die drei Handshakes der TCP-Protokolls (sieht Abbildung 5.7).

55	22.288290	192.168.4.2	192.168.4.1	TCP	66	60942 → 4210 [SYN] Seq=0 Win=17520 Len=0 MSS=1460 WS=256 SACK_PERM
56	22.290342	192.168.4.1	192.168.4.2	TCP	62	4210 → 60942 [SYN, ACK] Seq=0 Ack=1 Win=2144 Len=0 MSS=536 SACK_PERM
57	22.290397	192.168.4.2	192.168.4.1	TCP	54	60942 → 4210 [ACK] Seq=1 Ack=1 Win=17520 Len=0

ABBILDUNG 5.7: drei Handshakes der TCP-Protokolls

Dies zeigt, dass die TCP-Stream-Verbindung funktioniert. Daran kann man auch erkennen, dass die IP-Adresse des Handys 192.168.4.2 lautet. Danach können die Daten mithilfe von TCP-Paketen weitergeleitet werden (siehe Abbildung 5.8 und Abbildung 5.9)

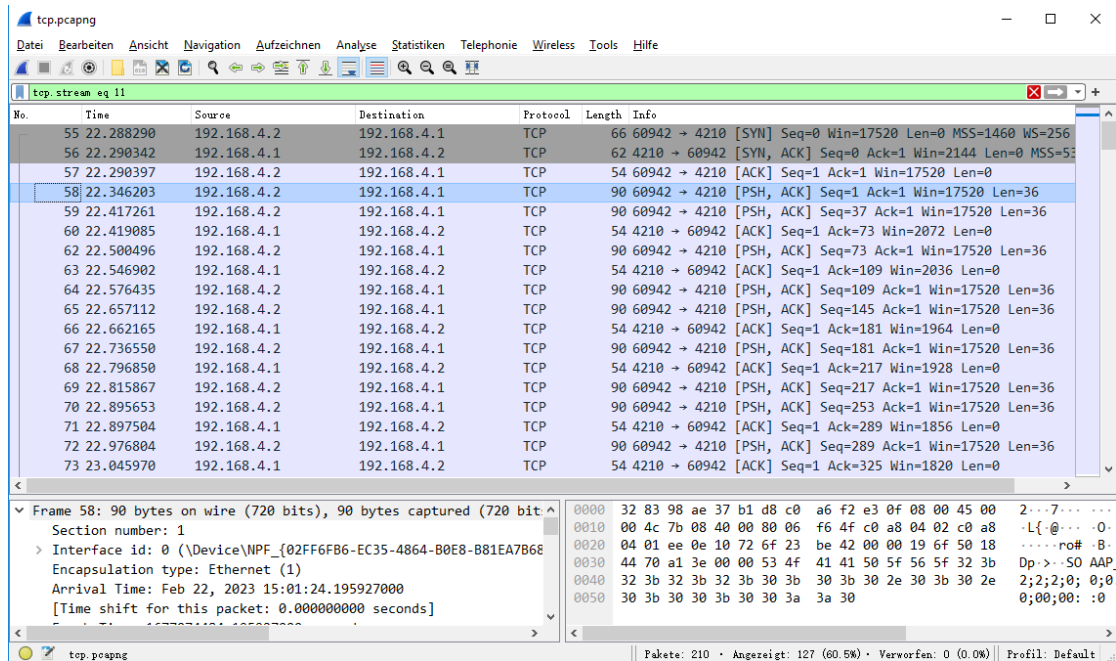


ABBILDUNG 5.8: TCP-Pakete mit nützliche Daten

Wie in Abbildung 5.9 dargestellt, die Länge des TCP-Header 20 Bytes beträgt. Source Port (Handy) ist 60942, Destination Port (Esp8266) ist 4210. Die Länge des *PDU*s beträgt 36 Bytes. *PDU* im Byte-Format ist *PDU*s im Byte-Format sind

```
53 4f 41 41 50 5f 56 5f 32 3b
32 3b 32 3b 32 3b 30 3b 30 3b 30 2e 30 3b 30 3e
30 3b 30 30 3b 30 30 3a 3a 30
```

Im *Folge TCP Stream* werden alle *PDU*s im Format ASCII angezeigt, die in diesem Zeitraum im TCP-Stream gesendet wurden(siehe Abbildung 5.10).

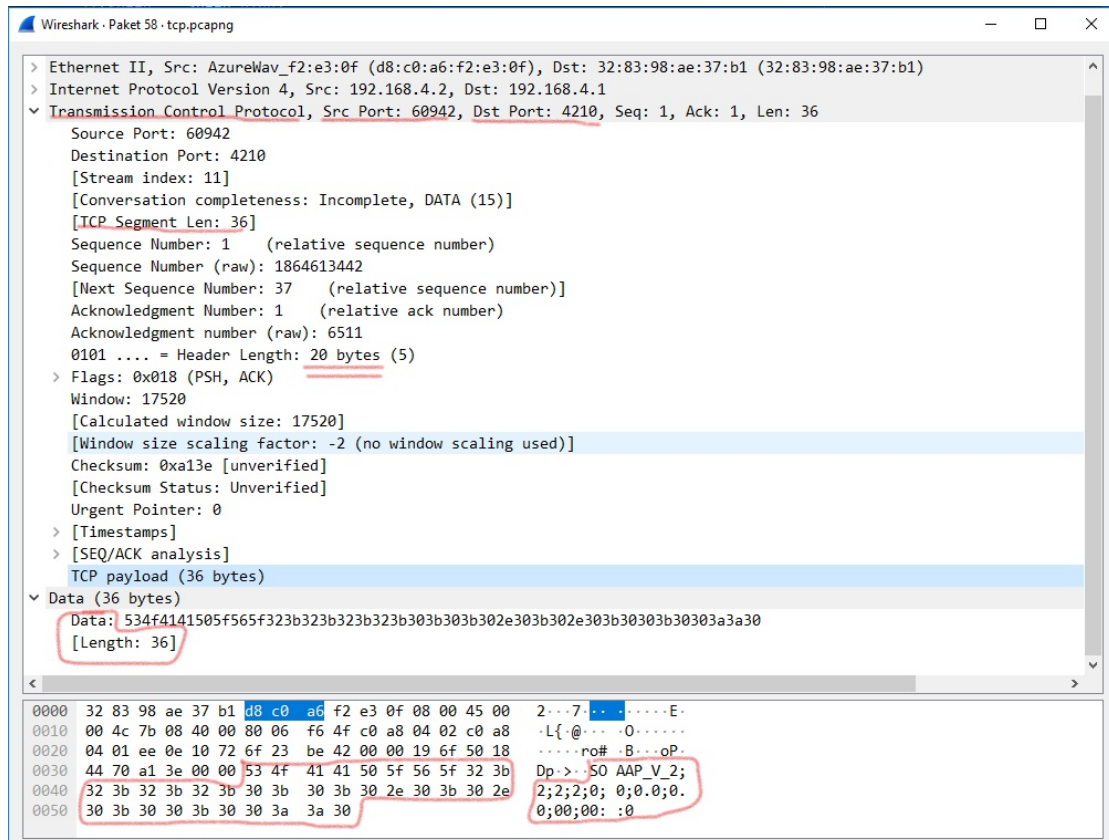


ABBILDUNG 5.9: TCP-Pakete Details

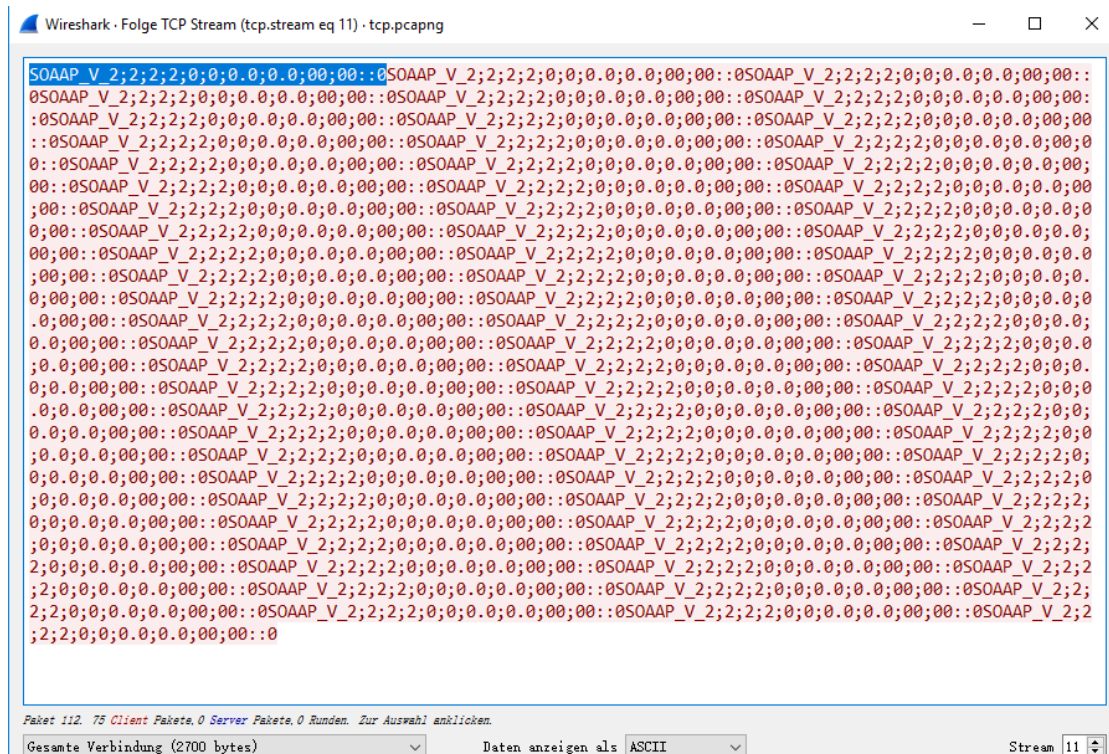


ABBILDUNG 5.10: Folge TCP Stream

5.1.6 Verlauf der Daten

Das folgende Verlaufsdiagramm zeigt den Ablauf der Datenübertragung. Wie im vorhergehenden Unterabschnitt wird der Esp8266 in diesem Unterabschnitt nicht berücksichtigt. Obwohl es eine Verzögerung bei der Weiterleitung der Nachrichten an den Esp8266 gibt, ist diese Verzögerung messbar und kann linear berechnet werden, sodass sie bei der Analyse des Gesamtverlaufs vernachlässigt werden kann.

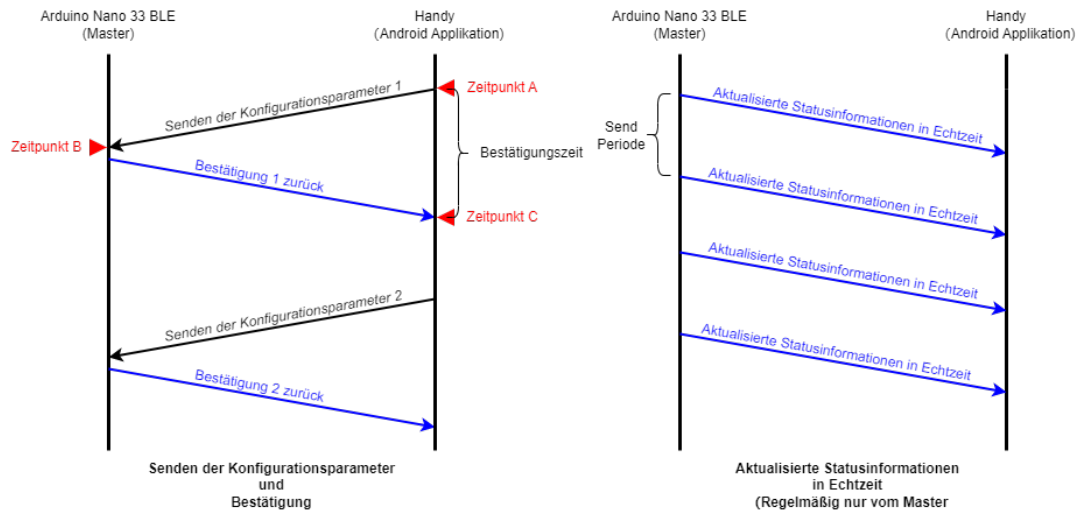


ABBILDUNG 5.11: Verlauf der Daten zwischen dem Master und der Android-Applikation

Der Verlauf kann in zwei Teilen untersucht werden: Im ersten Teil (links) sendet der Android eine Nachricht an den Master und der Master antwortet auf diese Nachricht, im zweiten Teil (rechts) sendet der Master regelmäßige Nachrichten an den Android. Die in beiden Verläufe verwendeten Telegramme sind die im vorherigen Unterabschnitt 5.1.5 erläuterten Telegramme und werden durch den *msgHeader* von *PDU* voneinander unterschieden.

Teil I (links): Zum Zeitpunkt A wird ein Telegramm vom Handy gesendet (ausgelöst durch Drücken eines Buttons in der Android-Applikation) und zum Zeitpunkt B vom Esp8266 empfangen. Anschließend wird ein Prozess im Master ausgeführt, um das Telegramm zu dekodieren und die empfangenen Daten in den entsprechenden Parameter zu speichern. Wenn der Prozess beendet ist, sendet der Master ein Telegramm an die Android-Applikation. Dieses Telegramm teilt dem UI mit, ob die Konfiguration im Master erfolgreich ist oder nicht. Schließlich zeigt die Benutzeroberfläche dem Benutzer die Ergebnisse der Konfiguration an. Die Zeit, die zwischen dem Drücken der Buttons und

der Anzeige der Meldung auf der Benutzeroberfläche vergeht, ist die Bestätigungszeit (siehe Abbildung 5.11). Da die Kommunikation zwischen dem UI und dem SOAAP-System keine besonders hohe Aktualität und Synchronisation erfordert, ist die Länge der Bestätigungszeit in dieser Arbeit nicht von vorrangiger Bedeutung.

Teil II (rechts): Unabhängig vom ersten Verlauf wird der zweite Verlauf ein Telegramm vom Master an die Android-Applikation in einer festgelegten Periode gesendet. Das Telegramm enthält Informationen wie den aktuellen Akkustand des Slaves, die Messwerte des Eulerwinkel usw. Außerdem ermittelt Android-Applikation den Verbindungsstatus der TCP-Verbindung mithilfe dieses periodischen Telegramms. Wenn eine bestimmte Zeit lang keine Nachricht empfangen wird, wird die Verbindung als getrennt angezeigt.

In dieser Arbeit wird nur der Verlauf der Sendung von Handy nach dem Master implementiert (Linie mit schwarzem Pfeil). Der Rest ist die Gedanken der zukünftigen Entwicklung (Linie mit blauem Pfeil).

5.2 Android Applikation

In diesem Abschnitt geht es um die Programmierung und Entwicklung von Android-Applikation

5.2.1 Übersicht

Drei Java-Dateien wurden für diese Android-Applikation programmiert, *SOAAPClient.java*, *BottomSheetDialog.java* und *MainActivity.java*, sie haben die folgenden Aufgaben

SOAAPClient.java - ist eine Klasse, die die Parameter von SOAAP-UI enthält, die bietet auch die Funktion der Erstellung der TCP-Verbindung und der Sendung der Nachrichten mit TCP-Protokoll.

BottomSheetDialog.java - wird verwendet, um ein *Bottom Sheet* zu erstellen.

MainActivity.java - wird den Hauptbildschirm der Android-Applikation repräsentieren und andere Klassen sind hier aufzurufen.

5.2.2 Codeerklärung

5.2.2.1 SOAAPClient

Die Klasse ist in zwei Teilen zu gliedern, der erste Teil behandelt den Parameter des SOAAP-UIs, der zweite Teil die TCP-Kommunikation.

Datensatz:

Zunächst sollten die Datentypen in der Klasse *MeasMuse* auch in der Klasse *SOAAPClient* mit Java definiert werden (nämlich *Meas2Midi*, *MeasMap* und *MidiOpMode*, siehe Unterabschnitt [4.5.3](#)). Die Datentypen mit Java sind

```
public enum Meas2Midi
{
    Nothing,          // keine Zielzuweisung
    NoteType,         // Zuweisung Notentyp (Laeng)
    NoteVal,          // Zuweisung NOTenwert (Hoehe)
    NoteVel           // Zuweisung Anschagstaeke
}

public enum MeasMap
{
    OneToOne,         // Direkte Weruebertragung
    BiLinear           // Linear auf Lienar
}

public enum MidiOpMode
{
    momIdel,
    momSequence,
    momRunDealt
}
```

```
}

```

Danach werden die Parameter *SoaapUiDatensatz* dargestellt, der den vorherigen Konfigurationsdaten (siehe 4.5.4) entspricht.

```
public class SoaapUiDatensatz
{
    // Parameter in SOAAPClient entspricht zu den Parameter in MeasMuse
    int      channel;

    float    rollwinkelAreaOffset;    //    offsetRoll
    float    rollwinkelAreaMin;       //    borderLowRoll
    float    rollwinkelAreaMax;       //    borderHighRoll

    float    pitchwinkelAreaOffset;   //    offsetPitch
    float    pitchwinkelAreaMin;      //    borderLowPitch
    float    pitchwinkelAreaMax;      //    borderHighPitch

    float    yawwinkelAreaOffset;     //    offsetYaw
    float    yawwinkelAreaMin;        //    borderLowYaw
    float    yawwinkelAreaMax;        //    borderHighYaw

                                //    MidiArea  midiArea[NrM2M];
    byte     midiAreaNoteTypeHigh;    //    midiArea[0].high
    byte     midiAreaNoteTypeLow;     //    midiArea[0].low
    byte     midiAreaNoteValHigh;     //    midiArea[1].high
    byte     midiAreaNoteValLow;      //    midiArea[1].low
    byte     midiAreaNoteVelHigh;     //    midiArea[2].high
    byte     midiAreaNoteVelLow;      //    midiArea[2].low

    Meas2Midi aimRoll;                //    aimRoll
    Meas2Midi aimPitch;               //    aimPitch
    Meas2Midi aimYaw;                 //    aimYaw
}
```

```
MeasMap    mapRoll;
MeasMap    mapPitch;
MeasMap    mapYaw;

MidiOpMode midiOpMode;      //    int midiOpMode

public void init()
{
    ...
}
}
```

Alle Daten sind lokale Variablen, auf die in anderen Klassen zugegriffen werden kann. Diese Daten können in anderen Klassen nicht direkt zugewiesen werden und sind nur über den Aufruf der Anwendungsfunktionen (siehe im Anhang) zugänglich. Alle Variablen der Klasse müssen initialisiert werden, nachdem eine Instanz dieser Klasse erzeugt wurde, daher sind ein Funktion zur Initialisierung aller Variablen in der Klasse enthalten.

```
public class SoaapUiDatensatz
{
    ...
    public void init()
    {
        channel = 0;

        rollwinkelAreaOffset = 50.0F;
        rollwinkelAreaMax = 90.0F;
        rollwinkelAreaMin = 10.0F;
        pitchwinkelAreaOffset = 50.0F;
        pitchwinkelAreaMax = 90.0F;
        pitchwinkelAreaMin = 10.0F;
        yawwinkelAreaOffset = 50.0F;
        yawwinkelAreaMax = 90.0F;
        yawwinkelAreaMin = 10.0F;
    }
}
```

```
        midiAreaNoteTypeHigh = (byte) 90;
        midiAreaNoteTypeLow = (byte) 10;
        midiAreaNoteValHigh = (byte) 90;
        midiAreaNoteValLow = (byte) 10;
        midiAreaNoteVelHigh = (byte) 90;
        midiAreaNoteVelLow = (byte) 10;

        aimRoll = Meas2Midi.Nothing;
        aimPitch = Meas2Midi.Nothing;
        aimYaw = Meas2Midi.Nothing;

        mapRoll = MeasMap.BiLinear;
        mapPitch = MeasMap.BiLinear;
        mapYaw = MeasMap.BiLinear;

        midiOpMode = MidiOpMode.momSequence;
    }
}
```

Beispiel für die Erstellung der Instanz und Initialisierung sind wie die Folge. In diesem Fall wird ein Array in der Type *SoaapUiDatensatz* erstellt, die Länge ist *nrChannel*.

```
public SoaapUiDatensatz[] soaapUiDaten;
soaapUiDaten = new SoaapUiDatensatz[nrChannel];
for(int i = 0; i < nrChannel; i++)
{
    soaapUiDaten[i] = new SoaapUiDatensatz();
}

for(int i = 0; i < ; i++)
{
    soaapUiDaten[i].init();
}
```

Die Anwendungsfunktionen werden in *set* und *get* unterteilt. *set* bedeutet die Zuweisung von Werten zu Parameter. Hier ein Beispiel mit der Variable *rollwinkelAreaOffset*. Die Set-Funktion hat zwei Argumenten, Das erste Argument *chn* bedeutet Kanal, das ist der Index des Arrays *soapUiDaten*. Da Midi 16 Kanäle bietet, ist das Argument *chn* von 1 bis 16 begrenzt. Das andere Argument *Value* ist der Wert, der der Variablen zugewiesen werden soll. Die vollständige Liste der Methoden siehe im Anhang.

```
public void setRollAreaOffset(int chn, float value)
{
    if(chn < 0) return;
    if(chn > 16) return;
    soapUiDaten[chn].rollwinkelAreaOffset = value;
}
```

TCP-Verbindung / Socket

Der Prozess, der die TCP-Kommunikation aufbaut, ruft die Methoden von der Klasse *java.net.Socket*), die Klasse *Socket* wird im Unterschnitt [3.1.4](#) erwähnt. Er sollte in der Lage sein, den Aufbau einer Verbindung, die Überprüfung der Verbindung und das Schließen der Verbindung sowie das Melden von Fehlern durchzuführen. Dieser Prozess sollte unabhängig im Hintergrund laufen. Die Aufgabe des Prozesses wird durch einen Zustandsmaschine realisiert, wie folgt sieht man eine beispielhaften Methodenaufrufen.

```
soapClient.run(frequency);
```

Das Zustandsdiagramm sieht man wie folgt.

Die Methode *run()* muss in einer festgelegten Periode über einen Timer aufgerufen werden, damit der Zustand des Zustandsmaschine ständig aktualisiert wird. Wenn die Applikation geöffnet, die Methode *run()* wird zum ersten Mal aufgeführt, wird es in den Zustand *Connect* gehen. Darin wird die Instanz von *Socket* erstellt. Die IP-Adresse und Der Port vom Zielserver werden hierzu wie folgt eingerichtet.

```
socket = new Socket(APConnectIP, APConnectPort);
```

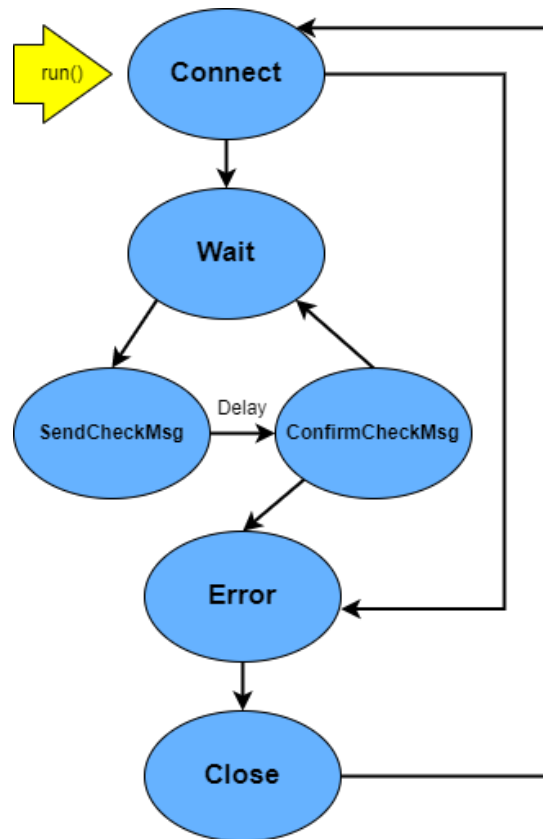


ABBILDUNG 5.12: Zustandsdiagramm der Erstellung der TCP-Kommunikation

Wenn der Socket erfolgreich geöffnet wurde, wird einer booleschen Variable (*isConnect*), die den Verbindungsstatus aufzeichnet, als *true* zugewiesen und der nächste Zustand wird auf *Wait* gesetzt. Sonst, wenn das Öffnen des Sockets fehlschlägt, wird ein Fehler gemeldet und der nächste Status wird auf *Error* gesetzt. Im *Wait* läuft einen Delay-Prozess. Nach einer bestimmten Wartezeit wird der nächste Status auf *SendCheckMsg* gesetzt. In *SendCheckMsg* wird eine Nachricht an den Server gesendet, um den Verbindungsstatus zu bestätigen, der nächste Zustand wird auf *ConfirmCheckMsg* gesetzt. Im *ConfirmCheckMsg* sollte eine *timeout* Funktion eingerichtet. Falls Android innerhalb der angegebenen Zeit eine Antwort vom Server bekommt, ist die Verbindung stabil, der nächste Zustand wird wieder auf *Wait* gesetzt. Andernfalls ist die Verbindung unterbrochen, springt der Zustand zu *Error*. Wenn der Zustand sich in *Error* befindet, was bedeutet, dass der Socket nicht erfolgreich geöffnet wurde oder die Verbindung auf irgendeine Weise unterbrochen wurde (z.B. der Server wurde heruntergefahren oder entfernt vom Android), wird der Zustand auf *Close* springen, darin wird Socket geschlossen. Mit dem Button *Reconnect* in der Benutzeroberfläche kann der Zustand wieder auf *Connect* zurückgesetzt werden, oder der gesamte Zustandsmaschine lässt sich durch

einen Neustart der Applikation zurückgesetzt.

Da in dieser Arbeit nur Nachrichten vom Android zum Server realisiert werden, wird nicht die gesamte Funktionalität des Zustandsmaschine implementiert, es sollte weiter entwickelt werden (siehe Abbildung 5.13).

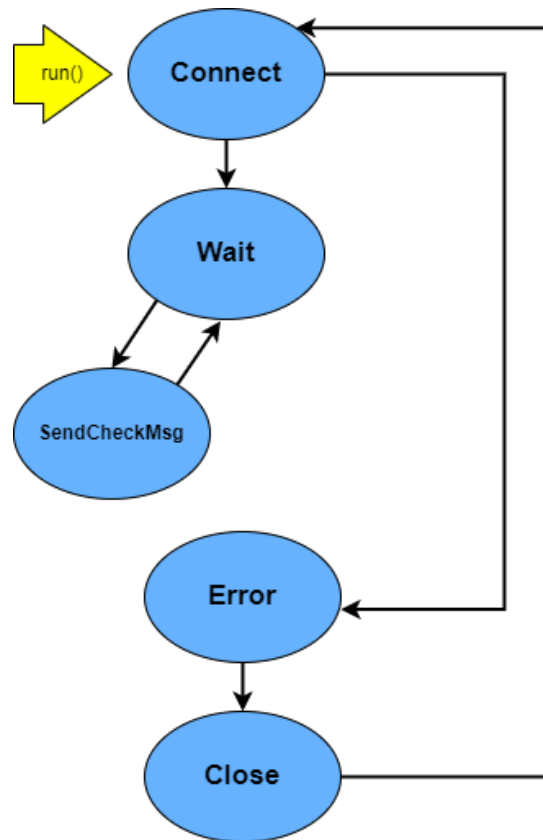


ABBILDUNG 5.13: Zustandsdiagramm der Erstellung der TCP-Kommunikation (implementierter Teil)

Create PDU

Das PDU im Telegramm, was im Unterabschnitt 5.1.5 erwähnt wird, kann man mit der Methode

```
soapClient.createPDU()
```

bekommen. Damit die Länge des Arrays in PDU vom Entwickler festgelegt werden kann, ohne dass die gesamte Methode umgeschrieben werden muss, wird die Methode durch eine Case-Anweisung implementiert. Über diese Methode werden alle Daten in einen Sendepuffer im String-Format geschrieben, der mit der Sendemethode vom Sockets versendet werden kann. Der Sendepuffer ist eine globale Variable.

Der ganze Prozess läuft wie folgt ab.

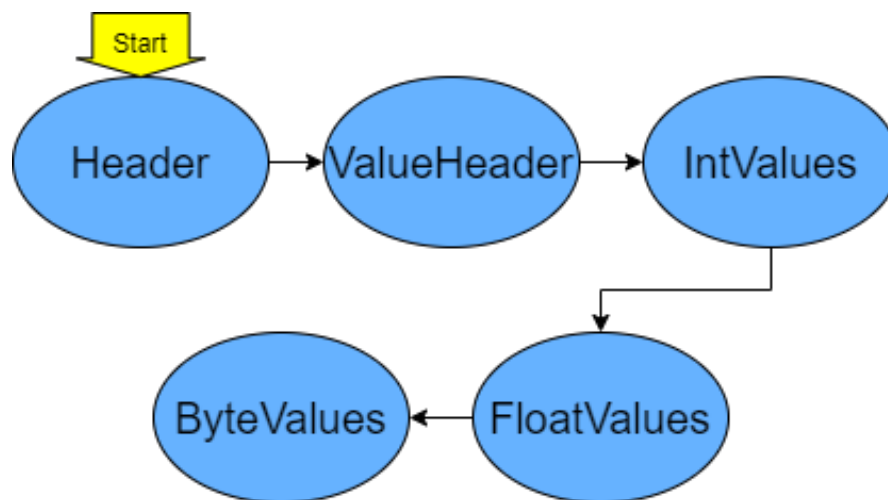


ABBILDUNG 5.14: Prozess der Methode *creatPDU()*

Im jeden Zustand hat den folgenden Prozess

- Header** - *msgHeader* wird in den Sendepuffer geschrieben.
- ValueHeader** - Die Länge der drei Arrays (*nrIntArray*, *nrFloatArray* und *nrByteArray*) werden in der Reihenfolge in den Sendepuffer geschrieben
- IntValues** - Alle Elemente des Integer-Arrays werden über einer for-Schleife in der Reihenfolge in den Sendepuffer geschrieben.
- FloatValues** - Alle Elemente des Float-Arrays werden über einer for-Schleife in der Reihenfolge in den Sendepuffer geschrieben.
- ByteValues** - Alle Elemente des Byte-Arrays werden über einer for-Schleife in der Reihenfolge in den Sendepuffer geschrieben.

Die drei Arrays sind:

```
private int[]      PduintArray;      // Integer-Array
private float[]    PdufloatArray;    // Float-Array
private byte[]     PdubyteArray;     // Byte-Array
```

Die Länge dieser Arrays (*nrIntArray*, *nrFloatArray* und *nrByteArray*) wird während der Initialisierung der Klasse *SOAAPClient* bestimmt. Die Methoden zur Initialisierung ist

```
public SOAAPClient(int nrInt, int nrFloat, int nrByte)
```

Die Daten werden in das String-Format konvertiert, wenn die in den Sendepuffer geschrieben werden. Diese Methode gibt ein *true* zurück, wenn alle Daten in den Sendepuffer geschrieben wurden. Ein Beispiel für ihre Verwendung wird wie folgt dargestellt.

```
boolean fin = false;

do{
    fin = createPDU();
}
while(fin!=false);
```

Set Array

Die Daten im ersten Unterabschnitt [5.2.2.1 SoaapUiDatensatz](#) müssen in Arrays im PDU gespeichert werden, bevor der PDU generiert werden können. Die dazu aufzurufende Methode lautet

```
soapClient.setPduDatensatz(chn)
```

chn bedeutet den Argument *Channel* (MIDI-Kanal), der wird auch von 1 bis 16 begrenzt. In dieser Methode werden drei Prozesse aufgeführt.

```

setPduIntArray (chn) ;
setPduFloatArray (chn) ;
setPduByteArray (chn) ;

```

Die drei Prozesse werden die Parameter (siehe *SoaapUiDatensatz* 5.2.2.1) mit ihren Datentypen in den entsprechenden Arrays speichern. In der folgende Tabelle 5.1 wird es formuliert, wie die Parameter in Integer-Array gespeichert werden.

Array[idx]	Parameter
PduIntArray[0]	Channel
PduIntArray[1]	aimRoll
PduIntArray[2]	aimPitch
PduIntArray[3]	aimYaw
PduIntArray[4]	mapRoll
PduIntArray[5]	mapPitch
PduIntArray[6]	mapYaw
PduIntArray[7]	midiOpMode

TABELLE 5.1: Formulierung des Integer-Array *PduIntArray*

An dieser Stelle sollen die folgenden Zuweisung von *PduIntArray[1]* bis *PduIntArray[7]* erläutert werden. Der Datentyp von *aimRoll*, *aimPitch* und *aimYaw* ist die Enumeration *Meas2Midi*, der kann in Integer umgewandelt werden (siehe Tabelle 5.2).

<i>Meas2Midi</i> Elemente	value
Nothing	0
NoteType	1
NoteVal	2
NoteVel	3

TABELLE 5.2: Umwandlung vom Datentyp *Meas2Midi* zu Integer

Gleichermaßen können *mapRoll*, *mapPitch*, *mapYaw* (in der Datentyp *MeasMap*) und *midiOpMode* (in der Datentyp *MeasOpMode*) in Integer umgewandelt werden.

<i>MeasMap</i> Elemente	value
OneToOne	0
BiLinear	1

TABELLE 5.3: Umwandlung vom Datentyp *MeasMap* zu Integer

In den folgenden Tabellen 5.1 wird es formuliert, wie die Parameter in Float-Array und Byte-Array gespeichert werden.

<i>MidiOpMode</i> Elemente	value
momIdel	0
momSequence	1
momRunDealt	2

TABELLE 5.4: Umwandlung vom Datentype *MidiOpMode* zu Integer

Array[idx]	Parameter
PdufloatArray[0]	rollwinkelAreaOffset
PdufloatArray[1]	rollwinkelAreaMin
PdufloatArray[2]	rollwinkelAreaMax
PdufloatArray[3]	pitchwinkelAreaOffset
PdufloatArray[4]	pitchwinkelAreaMax
PdufloatArray[5]	pitchwinkelAreaMin
PdufloatArray[6]	yawwinkelAreaOffset
PdufloatArray[7]	yawwinkelAreaMax
PdufloatArray[8]	yawwinkelAreaMin

TABELLE 5.5: Formulierung des Float-Array *PdufloatArray*

Array[idx]	Parameter
PdubyteArray[0]	midiAreaNoteTypeHigh
PdubyteArray[1]	midiAreaNoteTypeLow
PdubyteArray[2]	midiAreaNoteValHigh
PdubyteArray[3]	midiAreaNoteValLow
PdubyteArray[4]	midiAreaNoteVelHigh
PdubyteArray[5]	midiAreaNoteVelLow

TABELLE 5.6: Formulierung des Byte-Array *PduByteArray*

5.2.2.2 BottomSheetDialog

BottomSheetDialog erbt von der Klasse *BottomSheetDialogFragment*² und hat die Aufgabe, ein Bottom Sheet zu erstellen, in dem verschiedene UI-Komponenten platziert werden können, um die Funktionalität der UI zu erweitern.

5.2.2.3 MainActivity

Im *MainActivity* befinden sich vier Prozesse, *SoaapClientInit()*, *graphInit()*, *graphBetrieb()* sowie *timerInit()*. Die Prozesse *graphInit()* und *graphBetrieb()* sind für die Benutzeroberfläche relevant, *SoaapClientInit()* dient der Initialisierung der Klasse *SOAAP-Client*, im Prozess *timerInit()* ist eine Timer-Funktion, die einen Zustandsübergang einer Zustandsmaschine oder Funktionen auslöst.

²Mehre Details über *BottomSheetDialogFragment* sind auf dieser Webseite zu finden, <https://developer.android.com/reference/com/google/android/material/bottomsheet/BottomSheetDialogFragment>

Die Prozesse *graphInit()* und *graphBetrieb()* laufen im UI-Thread (direkt in der *MainActivity*). Da Socket nicht in UI-Thread geschrieben werden können, wie in dem Unterabschnitt 3.1.4 beschrieben, werden zusätzliche Threads benötigt, um den Socket-Prozess ausführen. Ein Diagramm 5.15 dafür sieht folgendermaßen aus.

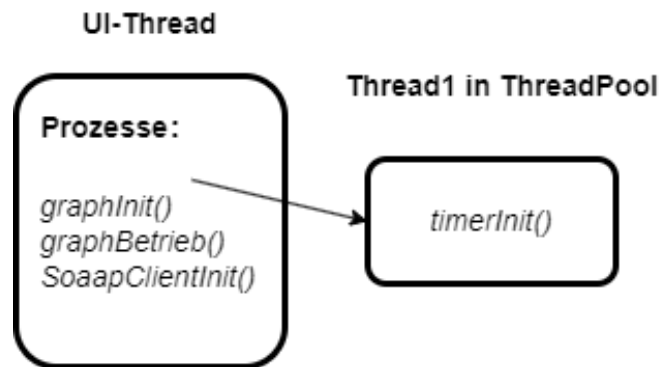


ABBILDUNG 5.15: Threads in *MainActivity*

graphInit() und **graphBetrieb()**

Im *graphInit()* werden alle Instanzen von UI-Komponenten initialisiert. Jede UI-Komponente steuert einen Parameter aus *SoaapUiDatensatz*. Die Callback-Methode der UI-Komponente wird dann in *graphBetrieb()* aufgerufen. Die folgende Tabelle 5.7 zeigt die verschiedenen UI-Komponenten und die von der gesteuerten Parameter.

UI-Komponente	Parameter
Rangeslider	rollwinkelAreaMin rollwinkelAreaMax pitchwinkelAreaMin pitchwinkelAreaMax yawwinkelAreaMin yawwinkelAreaMax
Slider	rollwinkelOffset pitchwinkelOffset yawwinkelOffset
Toggle Buttongroup	aimRoll aimPitch aimYaw mapRoll mapPitch mapYaw midiOpMode
Selector	channel

TABELLE 5.7: UI-Komponenten und die gesteuerte Parameter

SoaapClientInit()

Im *SoaapClientInit()* wird der Instanz der Klasse *SOAAPClient* initialisiert.

timerInit()

Im *timerInit()* läuft einen Timer Task mit der Timer-Periode 10 Millisekunden (mit der Frequenz 100Hz). Im Timer Task läuft die Zustandsmaschine, die in der Klasse *SOAAPClient* für die TCP-Kommunikation. Dazu wird die Methode

```
soapClient.run(Frequenz);
```

verwendet. Nach dem Start der Zustandsmaschine kann in der Callback-Methode der UI0-Komponente die Socket-*write()* und *read()* Methoden aufgerufen werden.

5.2.3 Darstellung des Layouts

In diesem Unterabschnitt ist die Einführung des Layouts der Android-Applikation. Das Layout kann in zwei Teile gegliedert werden: der erste Teil ist die Kontrollleiste, der zweite Teil ist die Schiebeleiste.

Die Kontrollleiste wird in der folgenden Abbildung 5.16 dargestellt.

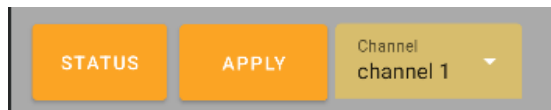


ABBILDUNG 5.16: Kontrollleiste im Layout der Android-Applikation

Darin befindet es sich zwei Bottöne und einen Selektor. Mit dem Botton *STATUS* wird ein Bottom Sheet geöffnet (siehe Abbildung 5.17). In diesem Stadium ist der noch leer, aber Entwickler können ihr weiter UI-Komponenten hinzufügen.

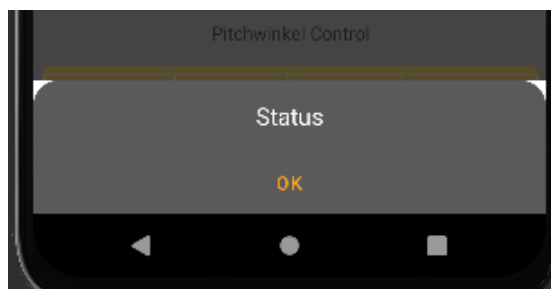


ABBILDUNG 5.17: Bottom Sheet im Layout der Android-Applikation

Mit dem Button *APPLY* wird alle Einstellung der Parameter bestätigt und ein Telegramm ausgesendet. Mit dem Selektor kann man MIDI-Channel auswählen. Auf der Schiebeleiste befinden sich alle UI-Komponenten zur Einstellung der Parameter. Die Beziehung zwischen den UI-Komponenten und den entsprechenden Parametern ist in der Tabelle 5.7 dargestellt.

Rangesliders und Sliders

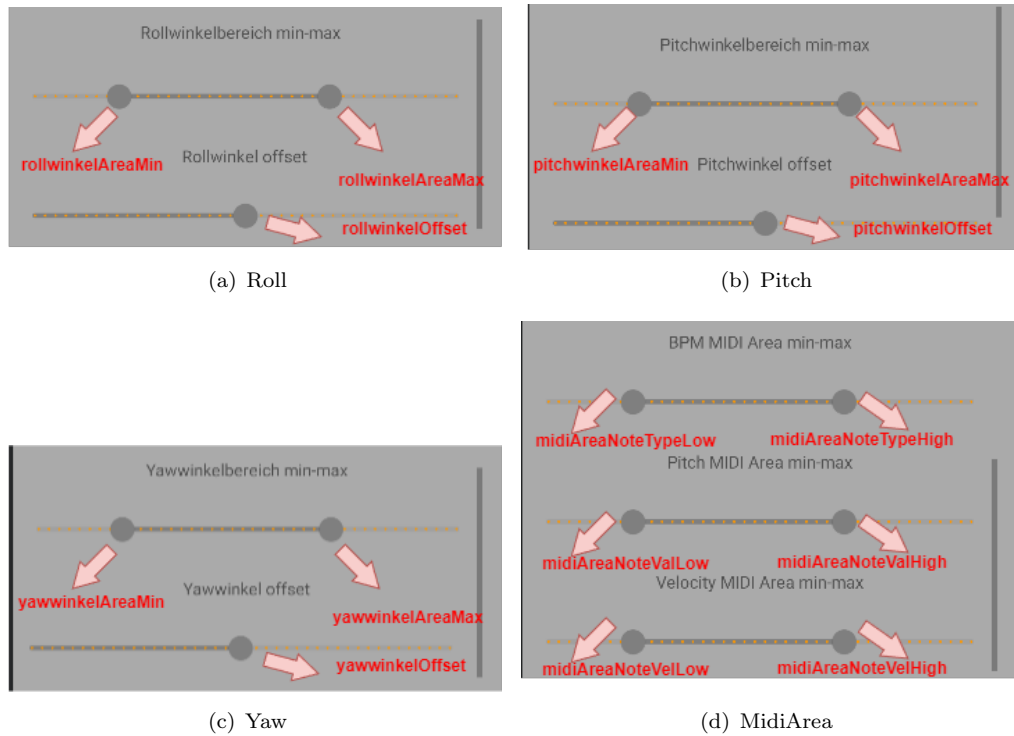


ABBILDUNG 5.18: Rangesliders und Sliders Layout

Toggle Buttongroup

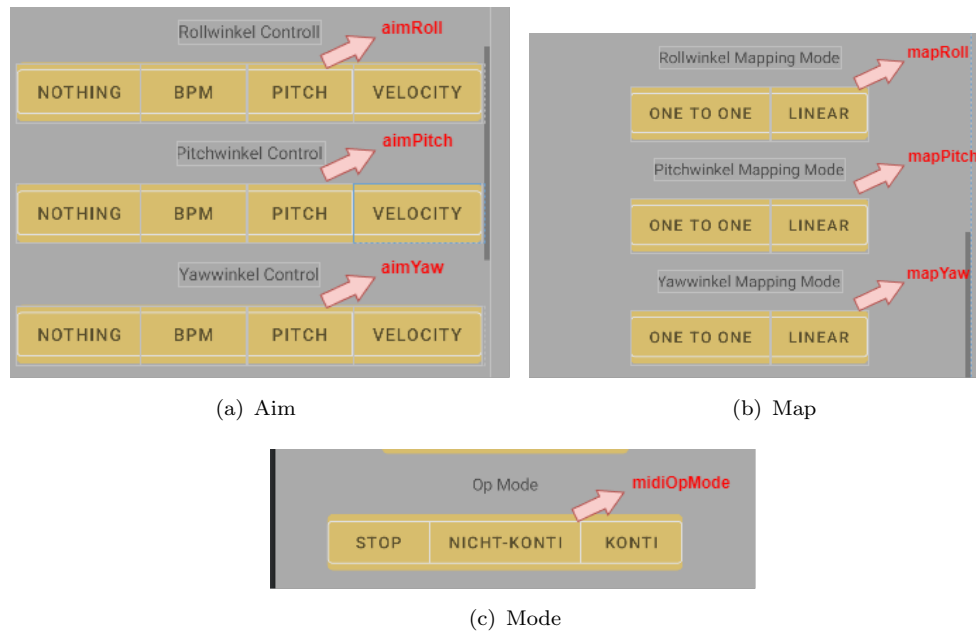


ABBILDUNG 5.19: Toggle Buttongroup Layout

5.3 Esp8266 Programmierung

In diesem Abschnitt geht es um die Programmierung und Entwicklung vom Esp8266.

5.3.1 Übersicht

Der *Esp8266* implementiert die Funktionen zum Einrichten eines WiFi-Servers, zum Empfangen von Messages (aus *Android-Applikation*), zum Dekodieren von Messages und zum Speichern von Daten.

5.3.2 Codeerklärung

Die Programmierung des *Esp8266s* kann in zwei Teilen beschrieben werden *setup()* und *loop()*.

setup()

Der *WiFiServer* wird in *setup()* eingerichtet. Es gibt zwei Modi, zwischen denen die Entwickler wählen können, *Station* und *Access Point* (über die Modi von *Esp8266* sieht

man im Unterabschnitt 3.2.3). Das Modul wird durch zwei Definitionen (*#define*) ausgewählt, sieht man wie folgt.

```
#define EspWifiNetwork    // Mode Station
```

oder

```
#define EspAccessPoint    // Mode Access Point
```

loop()

Der Prozessablauf wird im folgenden Diagramm dargestellt.

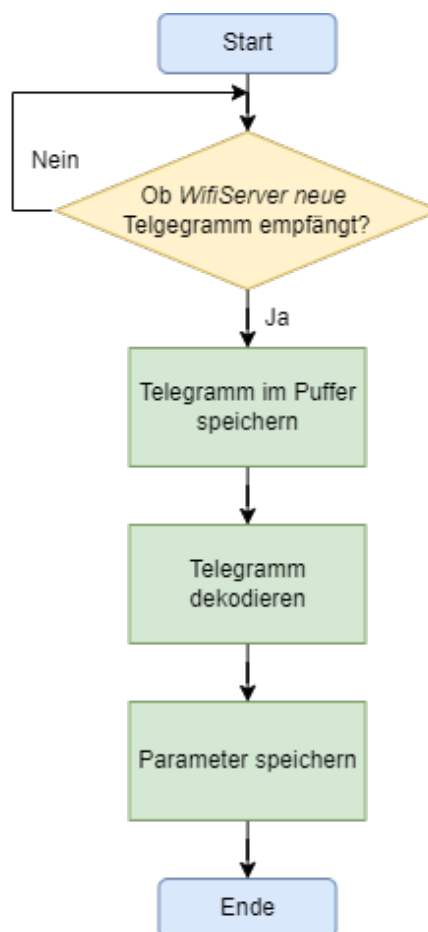


ABBILDUNG 5.20: Prozessablauf im `loop()` (*Esp8266*)

In der `loop()` sucht der *WiFiServer* ständig nach neuen empfangene Messages. Dieser Message wird in einem temporären Puffer mit der folgenden Methode gespeichert.

```
client.read(readBuf, inComingBytes);    // Lesen
```

Der Prozess geht dann in die nächste Phase über, in der die empfangene Message mit den folgenden zwei Methoden dekodiert wird.

```
int  parseMsg(char * msg, unsigned int msgLen);
```

```
int  storeDataMsg(char * msg, unsigned int msgLen);
```

Die Argumenten *msg* und *msgLen* sind der temporäre Puffer und die Länge dieses Puffers. Mit der Methode *parseMsg(msg, msgLen)* kann die Länge der drei Arrays (*nrIntArray*, *nrfloatArray* und *nrbyteArray*, siehe Abbildung 5.6) ermittelt werden, die Werte dieser drei Längen werden in drei globalen Variablen gespeichert. Nach Beendigung gibt die Funktion den Integer 1 zurück. Die Länge dieser drei Arrays wird in der Methode *storeDataMsg* benötigt. Da die empfangenen Telegramme im String-Format vorliegen, können die Daten in den drei Arrays (*Integer Array*, *Float Array* und *Byte Array*, siehe Abbildung 5.6), die in dem Telegramm enthält, mit Methode *storeDataMsg* in drei globale Arrays mit entsprechenden Datentyps gespeichert werden.

```
int      intArray[nrIntArray];  
double   floatArray[nrfloatArray];  
byte     byteArray[nrbyteArray];
```

Im letzten Prozess *Parameter speichern* wird die Methode *getValue* aufgerufen. Die Werte in den drei oben genannten globalen Arrays werden in den entsprechenden Variablen in der Klasse *MeasMuse* (siehe im Unterabschnitt 4.5.4) gespeichert. Mit der Debug-Methode

```
void printDatensatz(int chn)
```

kann man überprüfen, ob alle Variablen erfolgreich zugewiesen und gespeichert wurden.

Kapitel 6

Zusammenfassung und Ausblick

Die grundlegenden Aufgaben sind in dieser Arbeit abgeschlossen worden, einen Userinterface im Android für das Soaap-System zu entwickeln. Im Hinblick auf die Kommunikation wurde eine TCP-Kommunikation zwischen dem UI-System und dem Soaap-System implementiert (jedoch nur in der Richtung vom UI-System zum Soaap-System). Dabei wurde ein flexibles Telegramm definiert, das zukünftige Entwicklungen durch die Studierenden erleichtern soll. Zudem wurden einige Methoden zur Erstellung und Decodierung des Telegramms programmiert. Hinsichtlich der Benutzeroberfläche wurde ein einfaches, brauchbares Layout am Handy erstellt. Dem Benutzer steht die Möglichkeit offen, den räumlichen Winkelbereich entsprechend der individuellen Bewegungsfreiheit seiner Hände anzupassen oder eine Bewegungsrichtung festzulegen, die den Rhythmus (BPM), die Tonhöhe oder die Laufstärke steuert. Darüber hinaus kann der Benutzer sich auch den Klangmodus (kontinuierlich oder diskontinuierlich) entscheiden.

Auch hier gibt es noch viel Möglichkeiten für Verbesserungen. Einerseits muss die Kommunikation verbessert werden, z.B. durch die Realisierung der Zwei-Richtungen-Kommunikation. Und andererseits muss eine Rückmeldung über den Status der Kommunikation gegeben werden. Wenn zum Beispiel zwei Geräte zu weit voneinander entfernt sind, sodass die Verbindung instabil ist, oder wenn das Soaap-Geräte (*Arduino Nano 33 BLE* als Master) nicht eingeschaltet ist, kann die UI-Applikation am Handy kein Message senden, das muss der Benutzer wissen, was los ist. Darüber hinaus gibt es zahlreiche Möglichkeiten, die Funktionalität der Benutzeroberfläche zu erweitern.

Tatsächlich stehen heutzutage eine Vielzahl von technischen Optionen zur Verfügung, die es ermöglichen, die UI-Entwicklung nicht nur auf Handy mit Android oder IOS, sondern auch auf Computer, im Web oder kann das Soaap-Gerät (*Arduino Nano 33 BLE* als Master) mit einem Touchscreen ausgestattet sein, um eine benutzerfreundlichere Bedienung zu ermöglichen.

Kapitel 7

Anhang

Hier werden eine Liste über die Methoden (Anwendungsfunktionen) im Esp8266 Programm und Android Programm gestellt.

Android (SOAAPCleit.java)

```
void run(int secFactor)
// Zustandsmaschine für TCP-Verbindung starten

void setPduDatensatz(int chn)
// Parameter in den Arrays, die PDU braucht, speichern

boolean createPDU()      // PDU im String-Format erstellen
String getPDU()          // PDU bekommen
public void send()        // Msg senden

// Parameter set Anwendungsfunktionen
void setChannel(int chn)
void setAimRoll(int chn, Meas2Midi aim)
void setAimPitch(int chn, Meas2Midi aim)
void setAimYaw(int chn, Meas2Midi aim)
void setMappingRoll(int chn, MeasMap map)
```

```
void setMappingPitch(int chn, MeasMap map)
void setMappingYaw(int chn, MeasMap map)
void setOpMode(int chn, MidiOpMode opmode)
void setRollAreaOffset(int chn, float value)
void setRollAreaMin(int chn, float value)
void setRollAreaMax(int chn, float value)
void setPitchAreaOffset(int chn, float value)
void setPitchAreaMin(int chn, float value)
void setPitchAreaMax(int chn, float value)
void setYawAreaOffset(int chn, float value)
void setYawAreaMin(int chn, float value)
void setYawAreaMax(int chn, float value)
void setMidiNoteTypeAreaHigh(int chn, byte value)
void setMidiNoteTypeAreaLow(int chn, byte value)
void setMidiNoteValAreaHigh(int chn, byte value)
void setMidiNoteValAreaLow(int chn, byte value)
void setMidiNoteVelAreaHigh(int chn, byte value)
void setMidiNoteVelAreaLow(int chn, byte value)

// Parameter get Anwendungsfunktionen
int getChannel(int chn)
Meas2Midi getAimRoll(int chn)
Meas2Midi getAimPitch(int chn)
Meas2Midi getAimYaw(int chn)
MeasMap getMappingRoll(int chn)
MeasMap getMappingPitch(int chn)
MeasMap getMappingYaw(int chn)
MidiOpMode getOpMode(int chn)
float getRollAreaOffset(int chn)
float getRollAreaMin(int chn)
float getRollAreaMax(int chn)
float getPitchAreaOffset(int chn)
float getPitchAreaMin(int chn)
float getPitchAreaMax(int chn)
```

```
float getYawAreaOffset(int chn)
float getYawAreaMin(int chn)
float getYawAreaMax(int chn)
byte getMidiNoteTypeAreaHigh(int chn)
byte getMidiNoteTypeAreaLow(int chn)
byte getMidiNoteValAreaHigh(int chn)
byte getMidiNoteValAreaLow(int chn)
byte getMidiNoteVelAreaHigh(int chn)
byte getMidiNoteVelAreaLow(int chn)

// test Methoden
boolean Connect() // Socket öffnen
public void Disconnect() // Socket schließen
public void receive() // Msg empfangen
```

Esp8266

```
byte stringToByte(char *src); //String to Byte

int parseMsg(char * msg, unsigned int msgLen);
// Telegramm dekodieren

int storeDataMsg(char * msg, unsigned int msgLen);
// Daten im Temporäre Array speichern

void getValue(Posture2MidiPtr exampleParameter);
// Parameter anweisen

void printIdx();
void printArray();
void printDatensatz(int chn);
```

Literaturverzeichnis

- [1] Institut für angewandte Veranstaltungstechnik. Osi schichtenmodell, 2022. URL http://www.hbernstaedt.de/knowhow/steuersignale/steuersignale_05/Steuersignale05.html. [Online].
- [2] Edis Techlab. Wie funktioniert uart – einfach erklärt!, 2020. URL <https://edistechlab.com/wie-funktioniert-uart/?v=3a52f3c22ed6>. [Online; Stand 14. 11. 2023].
- [3] Arduino.cc. Arduino nano 33 ble, 2023. URL <https://store.arduino.cc/products/arduino-nano-33-ble>. [Online;].
- [4] WEMOS. Wemos d1 mini pro, 2023. URL https://www.wemos.cc/en/latest/d1/d1_mini_pro.html. [Online;].
- [5] Prof. Dr.-Ing. Robert Patzke. Steuerung optischer und akustischer Ausgaben für Performance-Künstler SOAAP. (German). 2022. URL https://moodle.hs-hannover.de/pluginfile.php/749086/mod_resource/content/6/PerfAkustikOptik01.pdf.
- [6] Android Wiki. Android sdk — android wiki,, 2017. URL https://www.droidwiki.org/w/index.php?title=Android_SDK&oldid=16116. [Online; abgerufen am 2. 3. 2023].
- [7] Wikipedia. Programmierschnittstelle — wikipedia, die freie enzyklopädie, 2022. URL <https://de.wikipedia.org/w/index.php?title=Programmierschnittstelle&oldid=229285306>. [Online; Stand 2. März 2023].

-
- [8] Wikipedia. Arduino (plattform) — wikipedia, die freie enzyklopädie, 2022. URL [https://de.wikipedia.org/w/index.php?title=Arduino_\(Plattform\)&oldid=228493581](https://de.wikipedia.org/w/index.php?title=Arduino_(Plattform)&oldid=228493581). [Online; Stand 2. März 2023].
- [9] Wikipedia. Esp8266 — wikipedia, die freie enzyklopädie, 2022. URL <https://de.wikipedia.org/w/index.php?title=ESP8266&oldid=223783958>. [Online; Stand 2. März 2023].
- [10] CLOUDFLARE. Was ist das osi-modell?, 2022. URL <https://www.cloudflare.com/de-de/learning/ddos/glossary/open-systems-interconnection-model-osi/>. [Online].
- [11] FS/community. Tcpip vs. osi: Was ist der unterschied?, 2022. URL <https://community.fs.com/de/blog/tcpip-vs-osi-whats-the-difference-between-the-two-models.html>. [Online].
- [12] Wikipedia. Socket (software) — wikipedia, die freie enzyklopädie, 2022. URL [https://de.wikipedia.org/w/index.php?title=Socket_\(Software\)&oldid=226224905](https://de.wikipedia.org/w/index.php?title=Socket_(Software)&oldid=226224905). [Online; Stand 2. März 2023].
- [13] Wikipedia. Wi-fi — wikipedia, die freie enzyklopädie, 2023. URL <https://de.wikipedia.org/w/index.php?title=Wi-Fi&oldid=231359043>. [Online; Stand 2. März 2023].
- [14] galaus. What is a wifi access point and what is it used for?, 2022. URL <https://www.galgus.ai/en/blog/wifi-access-points/>. [Online;].
- [15] Wikipedia. Java (programmiersprache) — wikipedia, die freie enzyklopädie, 2023. URL [https://de.wikipedia.org/w/index.php?title=Java_\(Programmiersprache\)&oldid=230307217](https://de.wikipedia.org/w/index.php?title=Java_(Programmiersprache)&oldid=230307217). [Online; Stand 2. März 2023].
- [16] Android Developers. AsyncTask, 2023. URL <https://developer.android.com/reference/android/os/AsyncTask>. [Online;].
- [17] Android Developers. Handlerthread, 2023. URL <https://developer.android.com/reference/android/os/HandlerThread>. [Online;].
- [18] Android Developers. Threadpoolexecutor, 2023. URL <https://developer.android.com/reference/java/util/concurrent/>

```
ThreadPoolExecutor#ThreadPoolExecutor(int,%20int,%20long,  
%20java.util.concurrent.TimeUnit,%20java.util.concurrent.  
BlockingQueue%3Cjava.lang.Runnable%3E). [Online;].
```

- [19] Google. Material components, 2023. URL <https://m3.material.io/components>. [Online;].
- [20] Esp8266 Arduino Core. Esp8266wifi library, 2023. URL <https://arduino-esp8266.readthedocs.io/en/latest/esp8266wifi/readme.html>. [Online;].
- [21] Esp8266 Arduino Core. Esp8266 access point, 2023. URL <https://arduino-esp8266.readthedocs.io/en/latest/esp8266wifi/soft-access-point-examples.html>. [Online;].
- [22] Esp8266 Arduino Core. Server, 2023. URL <https://arduino-esp8266.readthedocs.io/en/latest/esp8266wifi/server-examples.html>. [Online;].